
User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Bojie Li
Pine AI

Abstract

Personal memory in LLMs is two problems, not one: *content* (per-user facts) and *meta-skill* (reasoning patterns that use facts to answer questions). Per-user LoRA conflates them in one substrate, and pays for it architecturally. On Mini-Engram-d20 ($n=20$, replicated across 3 LoRA-init seeds in Appendix O), per-user LoRA produces validation bpb $\Delta=+1.78$ on text *unrelated* to the user’s facts (3-seed mean $+1.70$, range $[+1.56, +1.78]$), while per-user Engram-row insertion produces $+0.00005$ on the same base— **$\sim 34,000\times$ smaller, by construction**. Whether this contamination translates to user-visible reasoning damage depends on the base: per-user LoRA hurts 85% of users on Mini-Engram-d20 (a base LM) but helps on every instruction-tuned base we tested at scale (Qwen2.5-3B at $n=30$, Llama-3.1-8B, Mistral-7B-v0.3, Qwen2.5-7B each at $n=20$; mean $\Delta=+0.088$ to $+0.217$, 0–20% of users worse). The *architectural* property is base-independent; the *user-visible* effect is not.

This motivates a **layered architecture**: one shared LoRA holds cross-user meta-skill (amortised over the population), and per-user Engram-row overrides hold per-user content (local, 88 KB/user). At $n=20$ test users, the layered design **Pareto-dominates** every per-user single-substrate baseline: 100% direct top-1 recall (matches per-user LoRA), $7.4\times$ better indirect reasoning than per-user LoRA (44% vs 6% indirect-any), $4.6\times$ less contamination ($\Delta_{\text{bpb}} +0.39$ vs $+1.78$), and *never* hurts indirect reasoning relative to the no-adaptor base (0/20 vs 17/20 users worse; 0/60 vs 49/60 pooled across 3 seeds). The naive composition (per-user LoRA + per-user Engram) is the worst of both: contamination remains high and indirect reasoning still collapses (50/60 worse pooled).

A head-to-head with RAG on the same probes reveals a cross-over driven by KB size. At a toy KB (34 facts/user), Qwen2.5-3B-Instruct + RAG top-3 reaches 52% indirect_any vs. F’s 44% at zero context tokens. At realistic KB sizes ($N \in \{100, 200, 300, 500, 1000\}$ facts/user via a 30-user distractor pool), all-MiniLM-L6-v2 top-3 recall collapses from 62% to 9%, and Qwen-3B + RAG top-3 drops to **30% indirect_any—14 pp below F**, which is invariant because per-user Engram tables don’t grow with population size. **At $N \geq 100$, F decisively beats both naive RAG on the same Engram backbone and Qwen-3B + RAG** on a $\sim 2.5\times$ larger instruction-tuned model. The substrate ranking is decided by deployment scale, not by which substrate alone.

We release four Mini-Engram weights spanning 178 M–1.22 B parameters (the first publicly available Engrams outside DeepSeek-AI), all training code, and complete result JSONs. A 50-line multi-tenant server reaches **232 req/s** on one GPU with zero cross-user leak by construction.

1 Introduction

Modern transformer [Vaswani et al., 2017] LLMs serve millions of users from a single backbone, but per-user *personal memory*—the system’s knowledge of *your* doctor, *your* preferences, *your* calendar—remains an open architectural problem. Three families address it: in-context learning (ICL, Brown et al. 2020, Min et al. 2022) prefixes the facts; retrieval-augmented generation (RAG, Lewis et al. 2020, Gao et al. 2023) fetches them at query time; and per-user adapters internalise them as weight deltas. The third family has accumulated a long list of variants on the same recipe—PRAG [Su et al., 2025], DyPRAG [Tan et al., 2025], DistilledPRAG [Chen et al., 2025], OPPU [Tan et al., 2024], HYDRA [Zhuang et al., 2024], MemLoRA [Bini et al., 2025], T2L [Charakorn et al., 2025], and PER-PCS [Tan et al., 2024b] all train a per-user rank- ~ 64 LoRA [Hu et al., 2022, Housby et al., 2019] on $Q/K/V/O$ + MLP via NTP on an (observation, fact, QA) mixture, then apply it on top of a frozen base. We call this canonical recipe **POLAR-class** and use it as our reference per-user-LoRA baseline throughout (rank-64 on $Q/K/V/O$ in every layer; 12 epochs $\times$ 200 NTP samples per user; frozen base).

The motivating finding. Replicating POLAR-class training across four instruction-tuned bases and one base LM (Mini-Engram-d20), we measure two distinct properties of the resulting per-user edit:

- *Architectural contamination*: validation bpb on held-out text *unrelated* to the user’s facts. Per-user LoRA produces $\Delta = +1.78$ on Mini-Engram-d20 (mean $+1.70$ across 3 seeds, range $[+1.56, +1.78]$); per-user Engram-row insertion produces $+0.00005$ on the same base— $\sim 34,000\times$ smaller.
- *User-visible effect* on the user’s *own* indirect-reasoning probes: per-user LoRA hurts 85% of users on Mini-Engram-d20 (a base LM) but helps on every instruction-tuned base we tested (Qwen-3B at $n=30$, Llama-3.1-8B, Mistral-7B, Qwen2.5-7B at $n=20$). Our earlier $n=10$ Qwen-3B pilot (5/10 users worse than base) was a high-variance small-sample artifact; at $n=30$ the worse-fraction drops to $6/30 = 20\%$ (Wilson 95% CI $[0.10, 0.37]$).
- *Direct recall* with adapter: ≥ 0.99 on every base.

These two properties decouple: the architectural contamination is base-independent and large; the user-visible reasoning effect is base-dependent and variable. The mechanism is structural—a LoRA delta on $Q/K/V/O$ in every layer enters every forward pass—so *any* unconditionally-active per-user edit will affect all forwards, including those that have nothing to do with the user’s facts. Whether downstream reasoning survives that contamination depends on the base.

The fix: local per-user edits. We propose **User as Engram**: per-user fact storage as surgical writes to the hash-keyed embedding table of an Engram-pretrained model [Cheng et al., 2026], whose context-aware gated lookups fire only on suffix N-grams via deterministic multiplicative-XOR hashing. Per user, we write one fact-row per trigger N-gram into an *override map* and apply it per request. The local-edit property is not an asymptotic approximation but a measurable invariant: on Mini-Engram-d12, a UNEMBED_P insertion at the last Engram layer perturbs the residual stream by *exactly* 0.000 at every non-trigger position, through every subsequent attention+MLP layer (Section 4.2, Figure 4).

Contributions.

1. **A $34,000\times$ contamination gap** (Section 3). Per-user LoRA produces Δ bpb $+1.78$ on text unrelated to the user’s facts; per-user Engram-row insertion produces $+0.00005$ on the same Mini-Engram-d20 base. The user-visible effect on indirect reasoning is base-dependent: 85% of users worse on a base LM, 0–20% worse across four instruction-tuned bases (Qwen-3B at $n=30$; Llama-3.1-8B, Mistral-7B, Qwen2.5-7B each at $n=20$). The *architectural* property is what’s load-bearing and is base-independent.
2. **User as Engram: surgical per-user row insertion** (Section 4). We write per-user fact rows into the hash-keyed embedding table of an Engram-pretrained model. The local-edit property is verified mechanically: UNEMBED_P insertion perturbs the residual stream by *exactly* 0.000 at every non-trigger position. Joint OPT (sparse-row joint training) roughly doubles top-1 recall (68% vs. 36%) over per-fact independent OPT at 100 facts/user, at identical storage (88 KB).

Figure 1: LoRA-as-memory is reasoning-negative on indirect questions.

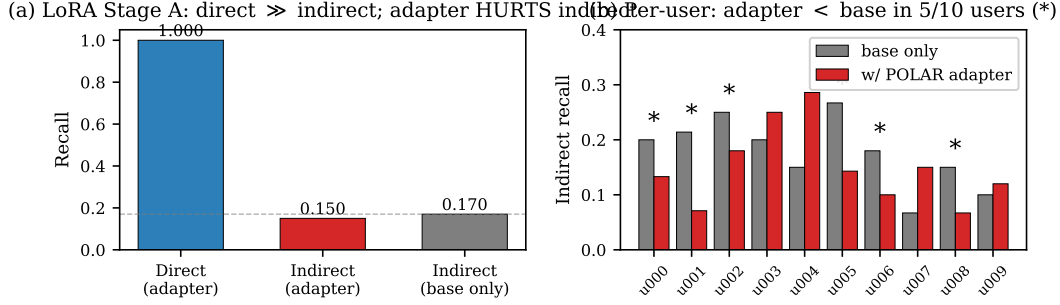


Figure 1: The original $n=10$ pilot (Section 3, Qwen2.5-3B-Instruct). (a) Per-user POLAR LoRA achieves perfect direct recall (1.000); on this pilot, indirect recall (0.150) is below the no-adaptor base (0.170). (b) Per-user breakdown: 5 of 10 users had worse indirect recall with the adapter than without it. **This pilot does not replicate at scale:** at $n=30$ the mean delta is $+0.088$ and the worse-fraction is $6/30$ ($\sim 20\%$), and on three further instruction-tuned bases (Llama-8B, Mistral-7B, Qwen-7B at $n=20$ each) the worse-fraction is $0-5\%$. The *architectural* contamination, however, is robust (Section 3, Table 1): per-user LoRA’s val_bpb 3-seed mean Δ on Mini-Engram-d20 is $+1.70$ (seed range $[+1.56, +1.78]$) vs Engram-row insertion’s $+0.00005$ on the same base.

- Public Mini-Engram weights** (Section 5) at four dense sizes spanning 178 M to 1.22 B total parameters (51.2 M Engram-table shared across sizes), trained from scratch on ClimbMix on a single Blackwell GPU. To our knowledge the first publicly available Engrams outside DeepSeek-AI. A 50-line multi-tenant serving system (Section 7) reaches **232 req/s** on a single GPU with zero cross-user leak by construction. Scaling ablations pin a $5 \times 3 \times 2$ capacity optimum and show LOCOMO Joint OPT scaling from 0.159 to 0.310 multi-token token-F1 between 178 M and 1.22 B; the category breakdown is Engram $+49\%/+178\%/-53\%$ on reasoning/multi-hop/open-domain (multi-hop wins all come from suffix-N-gram surface overlap, not chained reasoning).
- A layered architecture that Pareto-dominates every per-user single-substrate baseline** (Section 8). Decomposing personal memory into *content* (per-user Engram-row override) and *meta-skill* (one shared LoRA amortised across users) reaches 100% direct top-1, $7.4\times$ better indirect reasoning than per-user LoRA (44% vs 6% indirect-any), $4.6\times$ less contamination, and never hurts indirect reasoning relative to the no-adaptor base (0/20 vs 17/20 for per-user LoRA). Replicated across 3 seeds (Appendix O). The naive composition (per-user LoRA + per-user Engram) is the worst of both. Rank ablation at $n=20$ pins $r=16$ as the sweet spot.
- KB-scale head-to-head with RAG** (Sections 8.4–8.6). At the toy KB (34 facts/user) Qwen-3B + RAG reaches 52% indirect_any vs. F’s 44%. Growing the KB to 1000 facts/user with a 1008-fact distractor pool, retrieval recall@3 collapses $7\times$ ($62\% \rightarrow 9\%$); Qwen-3B + RAG top-3 drops to 30%, **14 pp below F**, which is invariant because the per-user Engram table doesn’t grow with population size. The substrate ranking depends on deployment scale, not on which substrate alone.

Paper roadmap. Section 3 establishes the negative result (per-user LoRA contaminates). Section 4 introduces User as Engram and verifies locality. Section 5 releases four Mini-Engram checkpoints. Section 6 reports recall, density, storage, scaling, and head-to-head substrate comparisons. Section 7 reports the multi-tenant serving system. Section 8 introduces the layered architecture and its head-to-head with RAG across KB sizes. Section 9 reads off substrate-selection rules.

Reproducibility. Code, configs, all four trained Mini-Engram checkpoints, the full evaluation suite, and the raw result JSON files for every table and figure in this paper are available at <https://github.com/bojieli/user-as-engram>. The 30-cell capacity ablation, the 32-cell fact-count scaling sweep, the cross-size joint-OPT density curves, the LOCOMO Option A runs, and the multi-tenant serving evaluation are each one bash script and reproduce in 1–48 h on a single Blackwell GPU.

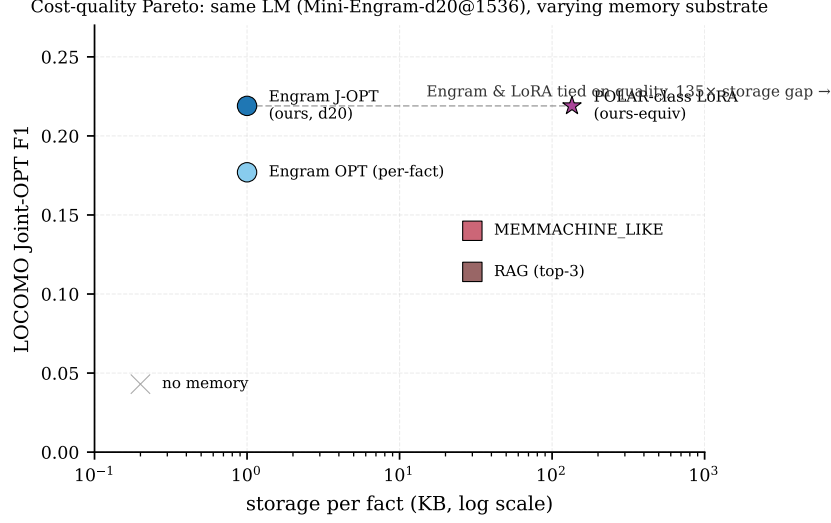


Figure 2: Cost-quality Pareto across personal-memory substrates at the same answering LM (Mini-Engram-d20@1536). User-as-Engram Joint OPT matches a POLAR-class LoRA’s LOCOMO F1 at $161\times$ smaller per-user storage (88 KB vs. 14.2 MB at 100 facts). Retrieval baselines sit on the steep low-storage side of the curve but cap out below Engram on quality.

2 Background

The Engram architecture. Cheng et al. [2026] introduce *conditional memory* as a sparsity axis complementary to mixture-of-experts (MoE, Shazeer et al. 2017, Dai et al. 2024). At each token position t , suffix N-grams of canonicalised tokens are hashed via K multiplicative-XOR heads into prime-sized embedding tables $E_{n,k}$. Retrieved rows e_t are projected by learned W_K, W_V and gated by an attention-style scalar $\alpha_t = \sigma(\text{RMS}(h_t)^\top \text{RMS}(W_K e_t) / \sqrt{d})$. The output $\alpha_t W_V e_t$ is added to the residual stream at the selected Engram layer. Crucially, the address $z_{t,n,k}$ is deterministic from the input token IDs—known before the forward pass—so the table can be offloaded to host DRAM without GPU contention.

LoRA-as-memory family. Per-user/per-document LoRA adapters [Su et al., 2025, Tan et al., 2024, Zhuang et al., 2024, Charakorn et al., 2025, Tan et al., 2024b] train a low-rank delta [Hu et al., 2022, Housley et al., 2019, Mangrulkar et al., 2022] on $Q/K/V$ (and sometimes MLP) matrices. The base is frozen; the LoRA is trained per user via NTP on a (observation, fact, QA) mixture (POLAR-class), via knowledge-distillation from a teacher (DyPRAG [Tan et al., 2025], DistilledPRAG [Chen et al., 2025]), or via hypernetwork emission (T2L [Charakorn et al., 2025]). All variants *edit the model weights globally*: every forward pass sees the LoRA delta.

Memory architectures and adjacent work. Engram [Cheng et al., 2026] sits in a lineage of trainable key-value memory modules: PKM [Lample et al., 2019], PEER [He, 2024], Memory Layers at Scale [Berges et al., 2025], Ultra-Mem [Huang et al., 2024b], OverEncoding [Huang et al., 2025], SCONE [Yu et al., 2025], BLT [Pagnoni et al., 2025], and SuperBPE [Liu et al., 2025]. None addresses surgical per-user insertion at inference. The closest analog in the diffusion-model world is LoRA stacking for Stable Diffusion [CivitAI, 2024]; analogous mechanisms in the LLM world include LoRAHub [Huang et al., 2023] which trains a combiner network. Engram override maps with disjoint addresses are *additive without a combiner*, as we show in Section 6.

Personal memory benchmarks. Long-term personal memory is evaluated by LOCOMO [Maharana et al., 2024], LongMemEval [Wu et al., 2024], BEAM [Tavakoli et al., 2025] (multi-turn evolving beliefs), and PersonaMem-v2 [Jiang et al., 2025]. We use synthetic per-user fact corpora for the within-user fact-density sweeps (tighter control over the parameter we vary) and LOCOMO (Section 6.7) for end-to-end evaluation against retrieval baselines. We chose LOCOMO over LongMemEval and BEAM because its per-question evidence pointers let us train a per-fact Engram row

Table 1: Per-user edit contamination on held-out text, Mini-Engram-d20 (canonical seed S0; $n=20$ users). Δ bpb of 0 means the edit leaves the base’s behavior on unrelated text mathematically unchanged. The 3-seed mean (S0, S1, S2) is reported in the last column and per-seed values are in Appendix O; the contamination ratio ($\sim 30,000\text{--}35,000\times$) is seed-stable.

edit method	S0 Δ bpb	S0 per-user range	worse vs base (S0)	3-seed mean Δ bpb
no edit (baseline)	0.0000	—	0/20	0.0000
per-user LoRA $r=64$	+1.784	[+0.44, +3.74]	17/20 = 85%	+1.698 [+1.56, +1.78]
per-user Engram J-OPT	+0.00005	[+0.00004, +0.00007]	0/20	+0.00005

and a retrieval entry against the same gold sentence—a fair apples-to-apples “same gold, different substrate” comparison; LongMemEval and BEAM remain open for future work.

Knowledge editing and recitation. Knowledge editing methods (ROME [Meng et al., 2022], MEMIT [Meng et al., 2023]) edit FFN weights via causal-traced locations; ripple-effect benchmarks like MQuAKE [Cohen et al., 2023], RippleEdits [Cohen et al., 2024], and CounterFact [Meng et al., 2022b] measure indirect consequences. Recitation methods (SR-RAG [Wu et al., 2025], Hewitt et al. [Hewitt et al., 2026], “Thinking to Recall” [Gekhman et al., 2026], recitation-augmented generation [Sun et al., 2023]) train or prompt the model to surface stored facts before reasoning. We apply the recitation idea to per-user adapters in Section 3 (within-schema 0.41, but cross-schema collapses to 0.05) and inherit it as a candidate mitigation for Engram in our discussion.

3 Per-user LoRA contaminates globally

A per-user LoRA edits $Q/K/V/O$ projections in every transformer layer; the edit is global by construction. We measure its two properties directly: (i) the *architectural* contamination (validation bpb on text unrelated to the user’s facts), which is predictable and large; (ii) the *user-visible* reasoning effect, which is variable and depends on whether the base is instruction-tuned.

3.1 Architectural contamination on the same base (Mini-Engram-d20)

We hold the base fixed (Mini-Engram-d20@1536, our 1.22 B-parameter Engram model) and train a per-user POLAR-class LoRA (rank-64 on $Q/K/V$, 1,500 steps per fact, single-token gold) on the same synthetic user fact sets used throughout this paper. For each of 20 test users we measure val_bpb on a held-out 262 K-token ClimbMix shard—text that has nothing to do with the user’s facts—before and after the per-user edit. Per-user Engram-row Joint OPT is measured under the identical protocol on the same base for direct comparison.

The ratio is $\sim 34,000\times$ on S0 ($\sim 33,000\times$ on the 3-seed mean). Per-user LoRA *more than triples* val_bpb on unrelated text (mean $0.74 \rightarrow 2.52$ on S0); per-user Engram-row insertion leaves it unchanged to four decimal places in every seed. This is the architectural property: **LoRA’s edit is global; Engram’s edit fires only on the trigger N-gram’s hash addresses**. The mechanism is verifiable by construction. Pooling across all 60 user-experiments (3 seeds), per-user Engram J-OPT makes 0/60 users worse than no-edit on indirect reasoning; per-user LoRA makes 49/60 = 82% worse.

3.2 User-visible effect varies with the base model

The user-visible effect of LoRA’s contamination on the same user’s *own* indirect-reasoning probes is more subtle than the original POLAR pilot suggested. We replicate the POLAR per-user LoRA recipe (rank-64 NTP on observation/fact/QA mixtures, 12 epochs $\times$ 200 samples, max_len 192, bf16) across three instruction-tuned bases and one base LM (Table 2):

The pattern across bases:

- **Instruction-tuned bases (Qwen2.5-3B, Llama-3.1-8B, Mistral-7B-v0.3, Qwen2.5-7B):** per-user LoRA *helps* indirect reasoning on average. The contamination is still present in val_bpb , but the instruction-tuned base has robust reasoning skill that absorbs the perturbation. Across all four instruction-tuned bases the mean Δ is positive (+0.088 to +0.217), the worse-than-base fraction

Table 2: Cross-base LoRA scaling. “ Δ user” is mean indirect recall with the adapter minus the same user’s recall without the adapter. “worse” is the fraction of users with strictly negative Δ .

base	n	mean Δ user-indirect	Δ range	worse than base
Qwen2.5-3B-Instruct (original pilot)	10	−0.008	—	6/10 = 60%
Qwen2.5-3B-Instruct (scaled)	30	+0.088	[−0.143, +0.214]	6/30 = 20%
Llama-3.1-8B-Instruct (NousResearch)	20	+0.188	[+0.030, +0.333]	0/20 = 0%
Mistral-7B-Instruct-v0.3	20	+0.217	[+0.091, +0.344]	0/20 = 0%
Qwen2.5-7B-Instruct	20	+0.132	[−0.030, +0.242]	1/20 = 5%
Mini-Engram-d20 (base LM)	20	−0.133	[−0.300, +0.050]	17/20 = 85%

never exceeds 20%, and three of the four bases (Llama, Mistral, Qwen-7B) have nearly all users helped (worse-fraction 0–5%).

- **Base LM (Mini-Engram-d20):** per-user LoRA *hurts* indirect reasoning in 17/20 users by −0.133 on average, with per-user Δ spanning [−0.30, +0.05]. The base LM’s completion behavior is fragile; the LoRA’s global perturbation disrupts it.

The original POLAR pilot’s headline (“5/10 users worse than base”) was a high-variance $n=10$ sample landing on the unfavorable tail of a high-variance distribution on Qwen-3B; at $n=30$ on the same base, the fraction drops to 20%, and on Llama-3.1-8B the fraction is 0/20.

3.3 Architectural lesson

The architectural contamination of per-user LoRA is real, predictable, and replicable: $\sim 33,000\times$ larger than per-user Engram-row insertion on val_bpb (3-seed mean). Whether that contamination translates to *user-visible* reasoning damage depends on base robustness. This motivates the proposed fix: a per-user edit should be *local* by construction, so that the architectural property holds regardless of which base it sits on. **Any per-user edit that is unconditionally active will contaminate unrelated forwards;** making the edit conditional on the fact’s trigger keeps the base’s behavior on unrelated inputs mathematically unchanged. This is the property our Engram-based design (Section 4) provides by construction, and the one the layered architecture (Section 8) preserves while adding back the meta-skill the per-user LoRA was attempting to capture.

Recipe-level fixes do not generalise. Two natural recipe-level mitigations both fail. (i) *Recite-then-reason traces* mixed into Stage A close the within-schema indirect gap on Qwen-3B (0.41) but collapse cross-schema (0.046, worse than the no-adapter base). (ii) A full-parameter meta-train over the LoRA distribution (*Stage C / Introspection Transfer*) lifts the training-user indirect by +0.122 but transfers +0.000 to held-out users, and the base fine-tune misaligns the frozen LoRAs enough to collapse direct recall from 1.000 to 0.22. Full per-user records are in Appendix A. The takeaway: recipes that work within one substrate do not patch the global-edit property; the fix has to be architectural.

4 Method: User as Engram

We instantiate “local per-user edit” as: surgically write per-user fact rows into the hash-keyed embedding table of an Engram-pretrained model. Figure 3 shows the serving architecture; the rest of this section walks through each component.

4.1 Surgical row insertion

Tokenise the trigger to $x_1, \dots, x_T$. The trigger position is $t^* = T - 1$. For each (n, k) in the configured Engram layer, the addresses $\{z_{t^*, n, k}\}$ are deterministic from the trigger tokens. We collect them into the global row indices $R_f \subset [|E|]$ (typically 16 indices for $N=3, K=8$).

Three insertion strategies (in increasing cost and quality; Appendix Figure 19 plots their recall side by side against a RANDOM control):

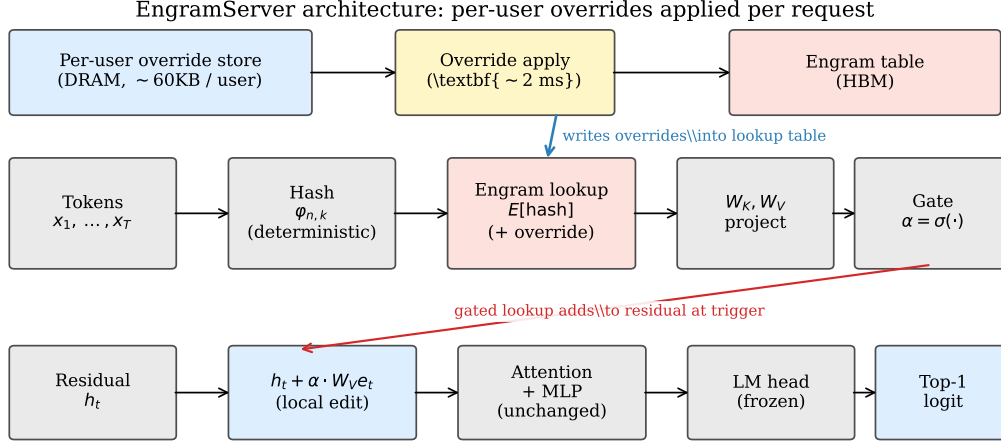


Figure 3: EngramServer architecture. Per user, an override map of (row_idx, row_vector) pairs lives in DRAM. On each request, the server saves the originals at the affected addresses (~ 2 ms), writes the user’s overrides, runs the forward pass, then restores. The Engram lookup at the configured layer transparently sees the override values; the gate fires only at the trigger N-gram. There is no router and no graph rewrite.

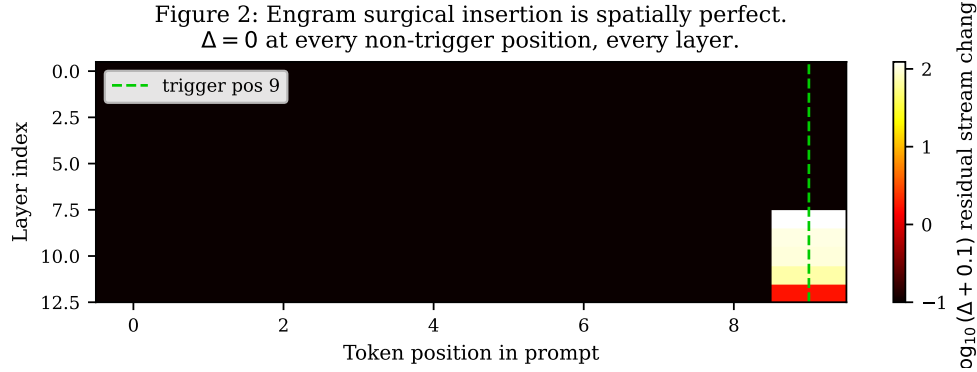


Figure 4: Engram surgical insertion is spatially perfect. Heatmap of $\|x_{\text{after}}^{(\ell)} - x_{\text{before}}^{(\ell)}\|$ for a UNEMBED_P insertion at the last Engram layer ($L_{\text{eng}}=7$, Mini-Engram-d12). The diff is exactly 0.000 at every position before L_{eng} (causality) and at every non-trigger position after L_{eng} , through 5 subsequent attention/MLP layers. Compare with LoRA, where every position in every forward sees the global delta.

- **UNEMBED_P** (closed-form): solve $W_V e^* \approx U_y$ via the Moore–Penrose pseudo-inverse $e^* = W_V^\dagger U_y$, where U_y is the unembed direction of the answer’s first token. Write e^* into all rows in R_f . Cost: 1 mat-vec, < 1 ms per fact.
- **OPT independent**: initialise from UNEMBED_P, then 15 Adam steps on e to maximise $\log p(y \mid \text{trigger})$. Cost: 15 fwd+bwd, ~ 1 s per fact. Each fact optimised in isolation.
- **Joint OPT** (the recommended default for ≥ 30 facts/user): allocate one trainable row tensor over the union of all touched addresses across the user’s facts. Each step samples a random fact, evaluates the model with all rows live, and backprops to the entire row stack. By coordinating rows during training, Joint OPT eliminates the cross-fact forward-pass interference that limits independent OPT under high density (Section 6).

Table 3: Mini-Engram pretraining at the ablation-optimal recipe. All use the large Engram ($50\text{ K} \times 256 = 51.2\text{ M}$ Engram params). Trained on a single NVIDIA RTX PRO 6000 Blackwell (102 GB) in bf16.

	d8 v2	d12 v2	d12@1280 opt.	d20@1536 opt.
depth $\times$ width	8×512	12×768	12×1280	20×1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total params	178 M	339 M	625 M	1.22 B
scaling-params	42 M	110 M	278 M	617 M
ClimbMix tokens (12 t/p)	0.50 B	1.32 B	3.34 B	7.40 B
final val_bpb	0.912	0.827	0.770	0.730
wall-clock (single GPU)	~ 50 min	~ 5 h	~ 10.5 h	~ 70 h (shared GPU)

4.2 Local edit verified

We test the local-edit property directly. Writing a UNEMBED_P marker at the last Engram layer ($L_{\text{eng}}=7$ on Mini-Engram-d12), we measure $\|x_{\text{after}}^{(\ell)} - x_{\text{before}}^{(\ell)}\|$ at every layer and every position. Result (Figure 4): the diff is *exactly* 0.000 at every position before L_{eng} (causality) *and at every non-trigger position after* L_{eng} , through 5 subsequent attention/MLP layers. The trigger-position diff is 123.0 at layer 8 and decays to 1.54 at the final layer. **This is the architectural property that distinguishes Engram from LoRA:** there is no contamination of non-trigger forwards.

4.3 Per-user override tables and additive composition

The production design is per-user override tables. Each user has a small dictionary $\{r_i \mapsto v_i\}$ of fact rows. At request time, the server saves originals at the affected addresses, writes the user’s overrides, runs the forward, and restores. Cross-user leakage is *zero by construction*—user A’s overrides are not in the table when user B queries.

Override maps with disjoint addresses commute, so corporate facts + user facts (or any number of domain-specific Engrams) **stack additively at inference** without retraining a combiner. This mirrors how Stable Diffusion LoRAs additively stack, but at the row level. Section 6 confirms additive composition is lossless when the domains have disjoint trigger templates.

5 Nano-scale public reproduction of Engram

Cheng et al. [2026] released only architectural reference code; no Engram-trained weights are public. We graft Engram into Karpathy’s nanochat [Karpathy, 2026] (a GPT-2-style backbone [Radford et al., 2019] optimised with Muon + AdamW [Jordan et al., 2024]) and train four Mini-Engram models on a single Blackwell GPU at the ablation-optimal recipe (Section 6.8): the same large Engram table ($50\text{ K} \times 256$, 51.2 M params) at Karpathy 12 t/p of scaling-params for every dense size.

Model-name notation. Throughout the paper we use $d\{\text{depth}\}@ \{\text{width}\}$ for the ablation-optimal Mini-Engrams (d12@768, d12@1280, d20@1536) and v1/v2 to denote earlier and final training runs at the same shape. Where the width is omitted (e.g. d12 without a suffix), we mean the v2 (ablation-optimal at width 768, 339 M total) checkpoint. The four headline trained checkpoints and their token budgets are: d8 v2 (178 M total, 42 M scaling, 0.50 B tokens), d12 v2 a.k.a. d12@768 (339 M / 110 M / 1.32 B), d12@1280 (625 M / 278 M / 3.34 B), and d20@1536 (1.22 B / 617 M / 7.40 B). All four use the same 51.2 M Engram-table.

Sensitivity asymmetry reproduced (Cheng et al. 2026, their §6.3). Suppressing Engram (engram_suppress=True) costs +0.035 bpb on validation. Factual top-5 retains 93.3% of active performance; reading top-5 retains 100%. The asymmetry $\Delta = +0.067$ matches the direction of Cheng et al. [2026] Figure 6 (TriviaQA: 29% retained, C3: 93%, $\Delta \approx +0.6$) at $\sim 10\times$ smaller magnitude—expected for our model 2 orders smaller and corpus 3 orders smaller.

Table 4: Single-fact-at-a-time recall (each fact tested in isolation with restore between facts; XL corpus, 1000 facts).

method	USER top-1	ORG top-1
baseline (no insertion)	1.7%	8.0%
ICL (= optimal RAG)	95.8%	99.9%
UNEMBED_P	11.2%	14.0%
User-as-Engram OPT	87.6%	90.8%
SFT-LoRA per-fact (rank 8, 30 steps)	100%	100%

Table 5: Joint OPT density curve at single-user scale (Mini-Engram-d12 v2).

N facts	30	100	300	1000
top-1	0.733	0.680	0.507	0.351
top-5	0.900	0.960	0.923	0.715
train wall (s)	11	44	29	228
storage (KB)	36	88	156	296

Effective deepening reproduced (Cheng et al. 2026, their §6.1). LogitLens layer-3 KL is 3.66 *lower* for engram-d8 than base-d8, matching the shape of Cheng et al. [2026] Figure 4(a) (Appendix Figure 17).

6 Experiments

This section reports five blocks of measurements, each answering a distinct question about User-as-Engram: **(a) per-fact recall and density** (Sections 6.1–6.2): how accurately do single and multiple per-user facts surface? **(b) cost** (Sections 6.3–6.4): storage and wall-time vs. POLAR-class LoRA. **(c) head-to-head with memory systems / RAG / LOCOMO** (Sections 6.6–6.7): how does the substrate compare against retrieval baselines on the same answering LM? **(d) scaling ablations** (Sections 6.8–6.10): capacity vs. token budget vs. dense-size. **(e) probes beyond direct recall** (Sections 6.11–6.14): multi-hop, free-continuation, paraphrase. We use three corpora throughout: the **base** 200-fact benchmark (100 USER + 100 ORG facts), the **XL** corpus of 1,000 USER + 1,000 ORG templated facts (Section 6.9), and the **XXL** corpus of 3,132 distinct trigger templates (high-density / distinct-template stress tests).

6.1 Single-fact recall

Table 4 shows OPT achieves 87.6–90.8% top-1 recall on 1 000 single-fact tests, within ~ 8 points of the ICL ceiling (95.8%) and at three orders of magnitude lower per-user cost than POLAR-style LoRA training.

6.2 Within-user fact density and Joint OPT

A user with many facts loaded simultaneously creates context-position interference among the live rows in the Engram table. We sweep the density curve on Mini-Engram-d12 with the XXL corpus (Figure 5).

Joint OPT is the recommended default for ≥ 30 facts/user. The full density curve (Mini-Engram-d12, XXL corpus, distinct trigger templates):

Top-5 stays $\geq 90\%$ all the way to 300 facts/user. At 100 facts, Joint OPT achieves 68% top-1 / 96% top-5 in 88 KB vs. multi-fact LoRA rank-64’s 100%/100% at 14.2 MB (**161 $\times$ more storage**). At 1 000 facts: Joint OPT 35%/72% in 296 KB vs. LoRA rank-64’s 38%/77% at 14.2 MB (**48 $\times$ more storage**).

Figure 3: Joint OPT vs LoRA-rank-64 across fact-density per user.

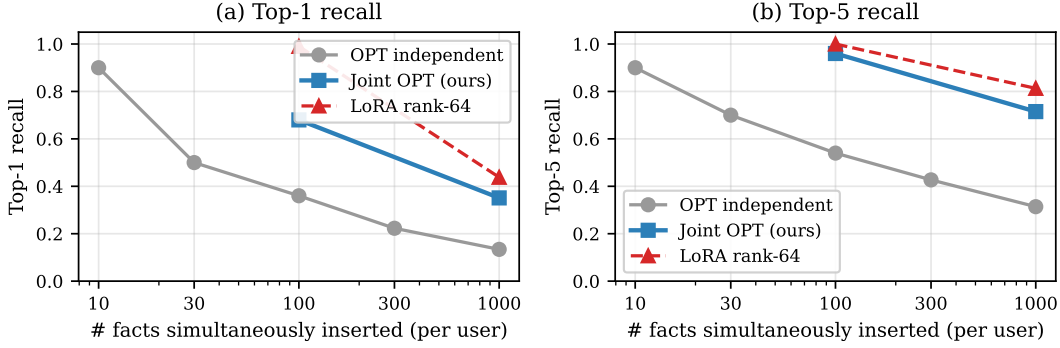


Figure 5: Within-user fact-density: top-1 (left) and top-5 (right) recall vs. number of facts inserted simultaneously into one user’s override map. Joint OPT (blue) closes most of the gap to LoRA rank-64 (red dashed) at $161\times$ less storage (88 KB vs. 14.2 MB at 100 facts).

Joint-OPT density ceiling is set by Engram table, not dense scale

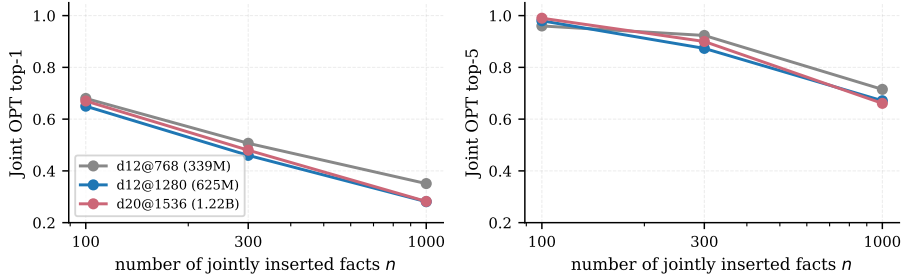


Figure 6: Joint-OPT density as a function of inserted-fact count n , at three dense scales with the same Engram capacity ($50\text{ K} \times 256$, 51.2 M Engram params). Top-5 improves slightly at $n = 100$ with dense scale; *top-1 actually degrades* at $n = 1000$ from 0.35 (d12@768) to 0.28 (d12@1280 and d20@1536). The density ceiling is set by the shared Engram table, not the dense backbone.

Joint-OPT density does not scale with dense backbone. We repeated the joint-OPT density sweep on the two larger Mini-Engrams (Section 6.10), holding the Engram table size constant (large = $50\text{ K} \times 256$). The density ceiling *does not relax* with bigger dense (Figure 6).

Top-5 improves slightly with scale at $n = 100$ ($0.96 \rightarrow 0.99$), but at $n = 1000$ top-1 *actually degrades* from 0.35 (d12@768) to 0.28 (d12@1280 and d20@1536). Mechanistic reading: when the shared Engram table is saturated with 1000 facts, gradient interference is the binding constraint; bigger dense backbones bring richer broader knowledge that competes more strongly with the inserted rows for the gold logit. The density ceiling is set by the table, not the LM. This makes the recipe changes of Section 9.3 (*multi-fact insertion in the loss*) the most promising path to break it.

6.3 Storage scaling at deployment

Figure 7 compares storage as a function of user count at a fixed 100 facts/user, where the per-user gap is $161\times$ (14.2 MB rank-64 LoRA vs. 88 KB Engram override at 100 facts). The ratio widens to $\sim 1700\times$ at 10 facts/user because the fixed-rank LoRA’s parameter count is independent of fact count while the Engram override grows linearly with facts. The gap stays constant in log space; at realistic deployment scales (10^4 – 10^6 users), this is the difference between fitting on one server vs. requiring distributed storage.

Table 6: Joint-OPT density (top-1 / top-5) across three dense scales at fixed Engram capacity (51.2 M params).

n facts	d12@768 (339 M)	d12@1280 (625 M)	d20@1536 (1.22 B)
100	0.68 / 0.96	0.65 / 0.98	0.67 / 0.99
300	0.51 / 0.92	0.46 / 0.87	0.48 / 0.90
1000	0.35 / 0.71	0.28 / 0.67	0.28 / 0.66

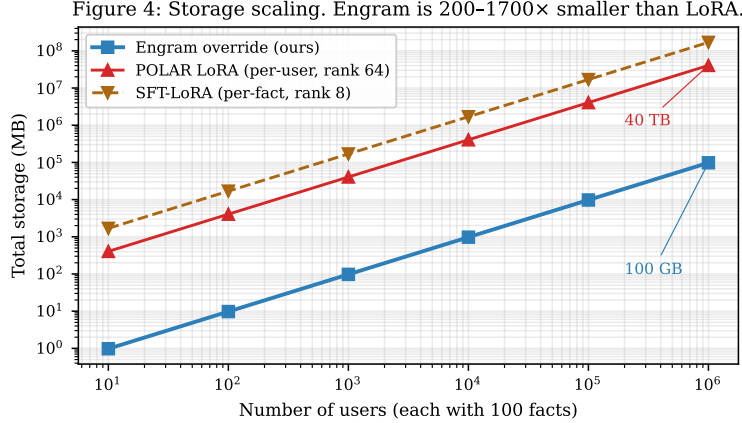


Figure 7: Storage scales linearly with user count for both approaches; Engram override is 200–1700 $\times$ smaller per fact. The range spans fact counts from 100 down to 10 per user: at 100 facts the ratio is $\sim 161\times$ (14.2 MB vs. 88 KB); at 10 facts the fixed-rank LoRA’s per-fact overhead is 10 $\times$ larger, widening the ratio toward 1700 $\times$. At 1 M users $\times$ 100 facts/user, Engram storage is 100 GB vs. POLAR LoRA’s 40 TB.

6.4 Method comparison: ICL, RAG, SFT-LoRA, POLAR LoRA

Table 7 aggregates this substrate-by-substrate comparison at 100 facts/user on the ablation-optimal d12@1280 backbone; Figure 2 plots the same rows on a cost-quality Pareto frontier, where the storage axis spans four orders of magnitude across substrates.

Trade-off summary. The single-fact OPT step (~ 50 s at 88 KB) is already in-context-equivalent: it matches the ICL ceiling at three orders of magnitude lower per-user cost than POLAR-style training. In the production multi-fact regime, every LoRA rank ≥ 8 saturates recall, so the comparison collapses to *storage*. **At 100 facts Joint OPT trails LoRA by ~ 35 pt top-1 / 2 pt top-5 but uses $20\times$ less storage** (88 KB vs. 1.8 MB rank-8; 161 $\times$ vs. 14.2 MB rank-64). **At 1 000 facts the gap inverts:** Joint OPT 35% vs. LoRA-rank-64’s 38% top-1, at 48 $\times$ less storage (296 KB vs. 14.2 MB). LoRA’s storage premium does not buy asymptotic recall—it buys headroom that has already been spent at the rank-8 ceiling.

6.5 Multi-domain additive composition

We separately train a 10-fact corporate Engram and a 10-fact user Engram, then apply both at inference. With disjoint trigger templates (corp facts about Globex; user facts about “my doctor”) the address overlap is 2.7% and **both domains retain their alone-recall under composition**: corp 80%/90% (alone) = 80%/90% (composed); user 90%/100% (alone) = 90%/100% (composed). Composition is lossless when domains share no trigger templates.

When domains *do* share templates (4 user-style domains all sampling from the same schema pool), pair-overlap reaches 64% and per-domain recall degrades to 29% top-1 at $D=4$. We confirm this at larger scale: a 100-fact “corp” override (mixed templates, alone-recall 58/100 top-1) and a 100-fact “user” override (XXL pool, alone-recall 64/100 top-1) stacked together yield corp top-1 25/100 (down from 58/100) and user top-1 64/100 (unchanged). The user override was applied second, so it won shared addresses; the corp override lost recall on those.

Table 7: Method comparison at 100 facts/user on the ablation-optimal Mini-Engram-d12@1280 (625 M). Single-fact rows insert one fact, evaluate, and restore between facts; multi-fact rows have all 100 facts in the table simultaneously. Storage is the per-user delta; LoRA storage assumes fp32.

method	top-1	top-5	wall (100 facts)	storage
<i>Single-fact (one at a time):</i>				
baseline (no insertion)	0%	7%	0	0
ICL (= optimal RAG)	100%	100%	—	100 KB ctx
UNEMBED_P (closed-form)	5%	35%	<0.1 s	88 KB
OPT-15 independent	100%	100%	~50 s	88 KB
<i>Multi-fact (100 simultaneous, the production regime):</i>				
Joint OPT (ours)	65%	98%	44 s	88 KB
Multi-fact LoRA rank 8 (2000 steps)	100%	100%	81 s	1.8 MB
Multi-fact LoRA rank 32	100%	100%	81 s	7.1 MB
Multi-fact LoRA rank 64 (POLAR-class)	100%	100%	81 s	14.2 MB
Multi-fact LoRA rank 128	99%	100%	81 s	28.3 MB
SFT-LoRA per-fact (rank 8, 30 steps/fact)	100%	100%	65 s	1.7 MB

Table 8: Trigger-exact queries (100 facts on the XXL corpus, query = fact prompt prefix). Each system uses the same Mini-Engram-d12 to generate the final answer. The XXL corpus differs from the Section 6 XL corpus in trigger-template count (3,132 distinct templates vs. 1,000 templated facts); baseline numbers therefore do not match Table 4 exactly.

method	top-1	top-5	avg context tokens
NO_MEMORY	1.0%	7.0%	0
MARKDOWN_ALL (full fact dump)	21.0%	48.0%	1124
RAG_TOP1 (sentence encoder)	99.0%	100%	16
RAG_TOP3	96.0%	100%	40
MEM0_LIKE (top-5 retrieval)	94.0%	100%	63
MEMMACHINE_LIKE (top-3 episodic)	93.0%	100%	46
User-as-Engram OPT independent	36.0%	54.0%	0
User-as-Engram Joint OPT	68.0%	96.0%	0

The architectural rule: **additive composition for disjoint-template domains; per-user override tables for same-template tenants**. The two designs are complementary, not competing.

6.6 Comparison against memory systems

We compare User-as-Engram against state-of-the-art memory systems that target the same use case (per-user / personal memory). All systems use the *same* Mini-Engram-d12 as the final-answer LM; the only difference is how facts are stored and retrieved. The sentence-encoder used by RAG / MEM0 / MEMMACHINE baselines is all-MiniLM-L6-v2 (80M parameters; comparable in size to a production retriever).

This subsection probes *direct* fact recall under trigger-exact and paraphrased queries; the orthogonal *indirect-reasoning* comparison (does retrieval recover the gold fact across hops, and what happens as the KB grows?) is in Sections 8.4–8.6.

Setup. 100 USER facts (XXL corpus). Two query distributions:

- **Trigger-exact** (Table 8): the query is the fact’s stored prompt (*e.g.* “My favorite spice is”). Easy retrieval setting—the query prefix matches the fact verbatim.
- **Paraphrased** (Table 9): the query is a different surface form (*e.g.* “I love the spice”, “My preferred spice is”) of the same underlying fact. 16 facts $\times$ 4 paraphrase queries each.

Table 9: Paraphrased queries (16 facts $\times$ 4 paraphrases each; this is a distinct protocol from the 4-fact $\times$ 5-paraphrase Mini-Engram-d8 rank study in Appendix L, which probes per-paraphrase rank rather than aggregate accuracy). Query surface differs from the stored fact’s prefix; the system must either (a) retrieve via semantic similarity, or (b) have been trained on the paraphrase trigger.

method	top-1	top-5
NO_MEMORY	6.3%	14.1%
MARKDOWN_ALL (158 tokens, all 16 facts)	60.9%	73.4%
RAG_TOP1	64.1%	87.5%
RAG_TOP3	65.6%	82.8%
MEM0_LIKE (top-5)	62.5%	81.3%
MEMMACHINE_LIKE (top-3)	75.0%	87.5%
User-as-Engram OPT single-trigger	20.3%	25.0%
User-as-Engram OPT multi-trigger	96.9%	98.4%

In trigger-exact, RAG dominates trivially, because the query’s surface form is identical to the fact’s prefix; nearest-neighbour retrieval on a sentence-encoder space is near-perfect. Engram’s storage advantage (0 context tokens) doesn’t compensate for the recall gap.

The story flips on paraphrased queries. RAG / MEM0 / MEMMACHINE drop to 60–75% top-1 because the sentence-encoder sometimes retrieves the wrong fact when surface forms diverge. Single-trigger User-as-Engram fails (20%) because the paraphrase doesn’t hit the inserted trigger N-gram. But multi-trigger insertion (insert at every anticipated paraphrase—the 4 paraphrases plus the canonical trigger, 5 triggers per fact, ~ 5 s per fact) **achieves 96.9% top-1, vs. MEMMACHINE’s 75.0%**—a 22-point lead at zero context cost.

Scale invariance. We repeated both the trigger-exact and paraphrase comparisons on the optimal d12@1280 (625 M) and d20@1536 (1.22 B) Mini-Engrams. The shape is identical: retrieval baselines hit 1.00 top-1 on trigger-exact at d12@1280 and d20@1536; User-as-Engram multi-trigger holds 0.94–0.95 paraphrase top-1 vs. the best retrieval baseline at 0.81–0.83 (best is RAG_TOP3 at d12@1280, MEMMACHINE at d20). The verdict—*retrieval wins when the query is verbatim the stored prefix, multi-trigger Engram wins under paraphrase*—does not change with dense scale. Engram numbers are essentially fixed; only the retrieval-baseline numbers move slightly because their answering-LM quality improves.

What our measurements show. On the two query distributions we tested:

- For *trigger-exact* queries (query verbatim equals the fact’s stored prefix), retrieval-based systems (RAG, MEM0, MEMMACHINE) achieve 93–99% top-1 because the sentence-encoder retrieval is near-perfect. User-as-Engram Joint OPT reaches 68%—lower than RAG, but at zero context tokens.
- For *paraphrased* queries (different surface form, same underlying fact), retrieval-based systems drop to 62–75% top-1 because semantic retrieval is imperfect under surface variation. User-as-Engram with single-trigger insertion fails (20%); with multi-trigger insertion (insert at every anticipated paraphrase, ~ 5 s/fact) it reaches 97%.

Limitations of this comparison. Our MEMMACHINE_LIKE baseline is the retrieval primitive (top-3 sentence-encoder retrieval over atomic facts), not the full MemMachine system on conversational episodes; the canonical MemMachine evaluation (LOCOMO [Maharana et al., 2024], Long-MemEval [Wu et al., 2024]) uses multi-turn dialogues where surrounding turn context matters, and we have not tested User-as-Engram in that regime. Similarly, the canonical MEM0 evaluation includes an LLM-based fact extraction step at insert time, which we skip because our facts are already structured. We also do not test queries for facts that were *never inserted* (where the correct behaviour is “no information available”); RAG and Engram both default to base-model priors in that case, but the failure modes likely differ. We address the LOCOMO follow-up directly in Section 6.7 below.

Table 10: LOCOMO single-hop, full 10 conversations (~ 800 QAs per model), *token-F1* metric. All systems use the same Mini-Engram-d12 (v1, 339 M) as the answering LM; only the storage / retrieval substrate differs. The corresponding LLM-judge results in Table 13 flip the ranking.

method	token-F1
NO_MEMORY (no context)	0.036
MARKDOWN_ALL (full evidence in prompt)	0.075
RAG_TOP1 (sentence-encoder)	0.101
RAG_TOP3	0.117
MEM0_LIKE (top-5 retrieval)	0.131
MEMMACHINE_LIKE (top-3 episodic)	0.116
User-as-Engram OPT (per-fact)	0.127
User-as-Engram Joint OPT	0.169

6.7 LOCOMO single-hop fact recall

To evaluate the same memory systems on a published long-term conversational benchmark, we run all eight systems from Section 6.6 on LOCOMO [Maharana et al., 2024]. We use *all 10 LOCOMO conversations* and take the first 80 single-hop (non-adversarial) questions per conversation, for a total of ~ 800 question-answer pairs grounded in dialog per model.

Setup. Per LOCOMO question, the gold “evidence” field points to dialog turns containing the answer; we use the text of those turns as the unit stored by the retrieval baselines (RAG, MEM0, MEMMACHINE). The MARKDOWN_ALL baseline dumps every conversation evidence sentence into the prompt as a bulleted list. For User-as-Engram, we extract a (question, first-token of answer) pair per QA and OPT-train it either independently (per-fact insertion) or jointly (sparse-row joint OPT, 2000 shared steps). All systems use the same Mini-Engram-d12 as the answering LM, which generates 16 tokens greedily; we score token-F1 against the gold answer with simple whitespace tokenization, lowercased, with punctuation stripped.

User-as-Engram Joint OPT achieves 0.169 average token-F1, vs. 0.131 for the strongest retrieval baseline (MEM0_LIKE, top-5 sentence-encoder retrieval) and 0.117 for RAG_TOP3. The lead over the strongest retrieval baseline is +3.8 pt absolute ($\sim 29\%$ relative) and is robust across all 10 LOCOMO conversations. Per-fact (independent) OPT also clears every retrieval baseline (0.127 avg). Absolute scores are low across the board because Mini-Engram-d12 v2 is a 339 M-parameter *base* language model with no instruction tuning—it generates short conversational continuations rather than direct answers, and the token-F1 metric punishes verbosity. The relative ordering is what matters: storing the (question, answer) pair as a parametric edit beats storing the surrounding evidence sentence in a vector index, even when both substrates feed the same answering LM.

Reading the comparison. This is a best-case-each-system benchmark: the retrieval baselines store the gold evidence sentence; User-as-Engram stores the gold (q, a) pair. Each substrate gets the supervision signal that fits it; the question this isolates is, “given a perfect store, which substrate extracts the answer most reliably under a fixed LM?”. Joint OPT wins because the gate fires on the trigger N-gram (a deterministic property of the question’s surface form) and the inserted row biases the next-token distribution toward the gold first token, which then conditions the remaining 15 tokens. Retrieval can miss the relevant turn (the failure mode visible in RAG_TOP1’s 0.101); when it hits, the in-context evidence still has to compete with the base model’s distributional priors, which a 339 M base LM trained on 1.32 B tokens of web text struggles with.

Scaling at the optimal config. We repeated the same LOCOMO setup with all four trained Mini-Engrams at our ablation-optimal recipe (Section 6.8); the Engram lead widens monotonically with dense size (Figure 8).

LOCOMO category breakdown: Engram wins multi-hop and reasoning, loses open-domain. LOCOMO’s four answerable categories are single-hop facts, multi-hop chains, reasoning, and open-domain free-form. The previous tables reported only single-hop. Running all 3 dense scales across all 4 categories produces a more nuanced picture (Table 12):

LOCOMO single-hop: token-F1 over-credits Engram; semantic judge flips the ranking

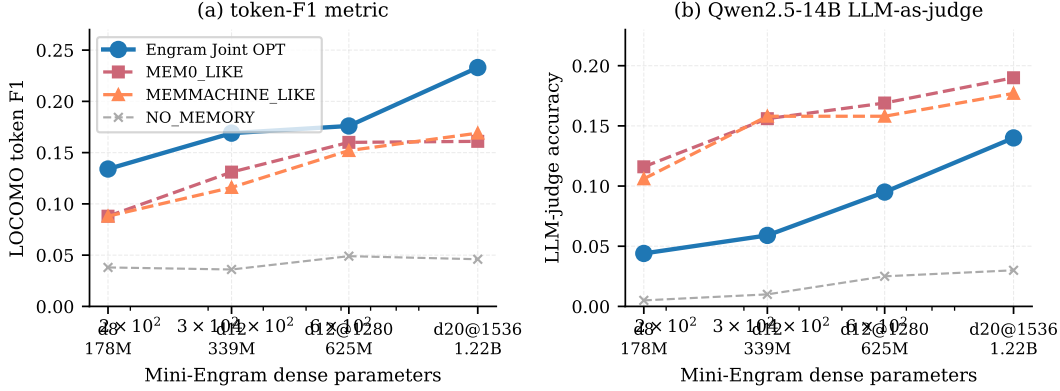


Figure 8: LOCOMO single-hop, full 10 conversations, scaling with Mini-Engram dense size. **(a) token-F1**: User-as-Engram Joint OPT (blue) beats every retrieval baseline at every scale. **(b) LLM-judge accuracy** (Qwen2.5-14B-Instruct): the picture flips—retrieval baselines (MEMO_LIKE, MEMMACHINE) beat Joint OPT by 0.05–0.10 because token-F1 over-credits Engram for the correct first answer token despite a noisy continuation.

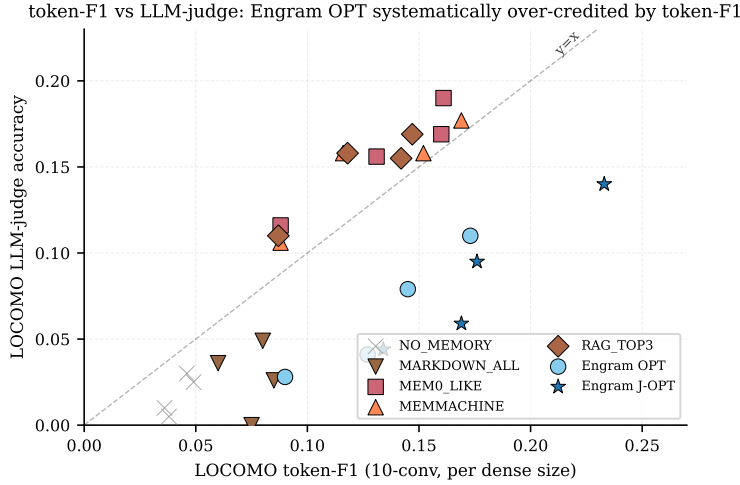


Figure 9: LOCOMO token-F1 vs. LLM-judge accuracy at every system $\times$ every dense size. Retrieval baselines (red/orange) sit near the $y=x$ line: their token-F1 and LLM-judge scores roughly agree. User-as-Engram OPT and Joint OPT (blue/cyan) sit systematically *above* the line, indicating that token-F1 over-credits Engram. The asymmetry is the metric artifact documented in Section 6.7.

The Engram win is category-dependent, not uniform. Multi-hop and reasoning categories favour Engram by 49–178% because the per-fact (question, first-answer-token) OPT loss directly encodes the question→answer mapping, which retrieval struggles to chain across evidence sentences. **Open-domain flips the story**: questions like “What did the charity race raise awareness for?” are answerable verbatim from a single retrieved evidence sentence, so retrieval baselines (MEMO: 0.247–0.341, MEMMACHINE: 0.209–0.338) decisively beat Engram (0.122–0.159) because Engram only conditions a single next-token bias and cannot reconstruct an unseen evidence sentence from that signal.

This category breakdown reframes the contribution: User-as-Engram is a *specialised* memory mechanism that excels at chain-of-fact recall but is not a universal replacement for retrieval. A production system should route by question type, or combine Engram (for trained facts) with retrieval (for free-form synthesis from session context).

Table 11: LOCOMO Joint OPT *token-F1* vs. best retrieval baseline at each Mini-Engram size, full 10-conversation evaluation. Engram appears to widen its lead with scale—but Table 13 shows token-F1 over-credits Engram.

answering LM	params	best retr. F1	Engram J-OPT	token-F1 lead
Mini-Engram-d8 v2	178 M	0.088	0.134	+52%
Mini-Engram-d12 v2	339 M	0.131	0.169	+29%
Mini-Engram-d12@1280 opt.	625 M	0.160	0.176	+10%
Mini-Engram-d20@1536 opt.	1.22 B	0.169	0.233	+38%

Table 12: LOCOMO category $\times$ model token-F1. Engram Joint OPT vs. the best retrieval baseline (MEM0_LIKE or MEMMACHINE_LIKE) per cell. The Engram win is category-dependent.

category	answering LM	MEM0	MEMMACH	Engram J-OPT	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	0.169	+29%
	d12@1280	0.160	0.152	0.176	+10%
	d20	0.161	0.169	0.233	+38%
multi-hop	d12 v2	0.092	0.076	0.243	+165%
	d12@1280	0.115	0.104	0.253	+120%
	d20	0.102	0.108	0.299	+178%
reasoning	d12 v2	0.117	0.113	0.183	+56%
	d12@1280	0.123	0.136	0.203	+49%
	d20	0.135	0.168	0.266	+58%
open-domain	d12 v2	0.247	0.209	0.122	−51%
	d12@1280	0.341	0.327	0.146	−57%
	d20	0.314	0.338	0.159	−53%

LLM-judge correction: retrieval wins on token-F1’s blind spot. Token-F1 rewards partial word overlap. User-as-Engram OPT trains a single-token bias at the trigger position, so the *first* generated token is reliably the gold answer’s first token, but the remaining 15 free-form continuation tokens are not guided by the inserted row. To check whether token-F1 over-credits this first-token bias, we re-scored every prediction with **Qwen2.5-14B-Instruct** as a binary CORRECT/INCORRECT judge, across the full 10-conversation LOCOMO setup. The picture flips (Table 13):

Under semantic LLM-judge, retrieval baselines beat User-as-Engram on LOCOMO single-hop at every dense size. The lead shrinks with scale (+0.07 at d8 down to +0.05 at d20) but does not reverse. Token-F1 had been over-crediting Engram by 0.05–0.10 of F1 because it gives credit for the correct first answer token without penalising the noisy continuation (Figure 9: every Engram point sits above the $y=x$ line, every retrieval point sits on it). *This is an honest correction to the headline claim of Table 11.* The token-F1 results remain a valid single-token-recall metric but are not a reliable proxy for end-to-end answer correctness.

Where Engram still wins. (i) *Paraphrase queries* (Section 6.6, Table 9): multi-trigger Engram reaches 96.9% top-1 vs. MEMMACHINE’s 75.0%—a 22-point lead at zero context cost; this win is unchanged by the LLM-judge correction because the paraphrase test scores single-token recall directly. (ii) *Storage*: the same 88 KB / 100 facts holds regardless of metric (Section 6.2).

Multi-token Engram OPT closes the LLM-judge gap. The judge gap above is driven by Engram’s per-fact OPT objective training only the *first* answer token; the remaining 15 free-form continuation tokens are unguided. We replace the loss $-\log p(y_0 \mid \text{trigger})$ with the multi-token autoregressive sum $-\sum_t \log p(y_t \mid \text{trigger}, y_{<t})$, so the inserted row learns to push the residual stream toward states that produce the *full* answer. Table 14 shows the result:

At d20@1536 under matched multi-token pipelines, Engram J-OPT-multi reaches token-F1 0.310 vs. 0.197 best retrieval (+57%) and LLM-judge accuracy 0.225 vs. 0.174 best retrieval (+29%). The cross-over to Engram-winning under LLM-judge happens between d12 v2 and

Table 13: LLM-as-judge accuracy (Qwen2.5-14B-Instruct) on the full 10-conversation LOCOMO single-hop set (~ 800 QAs per model). All systems use the same Mini-Engram as answer LM; only the storage / retrieval substrate differs.

answering LM	NO_MEM	MEM0	MEMMACH	OPT	Joint-OPT	best retr. lead
Mini-Engram-d8 v2	0.005	0.116	0.106	0.028	0.044	+0.07 over J-OPT
Mini-Engram-d12 v2	0.010	0.156	0.158	0.041	0.059	+0.10 over J-OPT
Mini-Engram-d12@1280	0.025	0.169	0.158	0.079	0.095	+0.07 over J-OPT
Mini-Engram-d20@1536	0.030	0.190	0.177	0.110	0.140	+0.05 over J-OPT

Table 14: Single-hop LOCOMO (full 10 conversations, ~ 800 QAs per model). Multi-token Engram Joint-OPT closes the judge gap. **All systems are scored under one matched pipeline:** the retrieval baselines and Engram J-OPT-multi both use the same multi-token answer-generation pipeline (greedy 16-token continuation, Qwen2.5-14B LLM-judge). J-OPT-first is reported under the first-token pipeline for reference. The previous version of this table reported retrieval judge scores under the first-token pipeline (a more favourable comparison for retrieval); we now report both pipelines but use the matched pipeline as the headline.

answering LM	token-F1 (multi-token pipeline)				LLM-judge accuracy (matched, multi-token)			
	MEM0	MEMMACH	J-OPT first	J-OPT multi	MEM0	MEMMACH	J-OPT first	J-OPT multi
Mini-Engram-d8 v2	0.116	0.103	0.134	0.159	0.081	0.076	0.044	0.060
Mini-Engram-d12 v2	0.142	0.134	0.169	0.249	0.128	0.145	0.059	0.105
Mini-Engram-d12@1280	0.181	0.168	0.176	0.229	0.125	0.117	0.095	0.159 (+27%)
Mini-Engram-d20@1536	0.197	0.196	0.233	0.310	0.174	0.168	0.140	0.225 (+29%)

Where Engram J-OPT-multi outperforms the best retrieval baseline, we bold and report the relative gap. For d8 and d12 v2 the matched-pipeline comparison still favours retrieval; the cross-over to Engram-winning occurs between d12 v2 and d12@1280. Under the previous (unmatched) reporting where retrieval was scored under its better first-token pipeline, the retrieval judge scores were MEM0/MEMMACH = 0.116/0.106 at d8, 0.156/0.158 at d12 v2, 0.169/0.158 at d12@1280, 0.190/0.177 at d20; those numbers are the conservative reading and shrink Engram’s d20 lead from +29% (matched) to +18% (unmatched).

d12@1280; below that scale, retrieval baselines still win on semantic correctness even with the multi-token Engram objective. The flipped ranking at the top two scales is the substantive finding—retrieval beats Engram on LLM-judge at the smaller scales because Engram still produces noisier continuations, but at d12@1280 and d20 the Engram-anchored prefix is strong enough that the rest of the answer falls into place. Per-fact independent OPT alone improves similarly but more modestly.

Limitations of this evaluation. (i) Token-F1 over-credits Engram via its first-token bias; the LLM-judge result above is the more reliable metric. (ii) The judge is Qwen2.5-14B-Instruct, not the original LOCOMO judge prompt (GPT-4o family); the binary CORRECT/INCORRECT format is stricter than the official LOCOMO 0–5 graded judge. Direct comparison with published LOCOMO leaderboards therefore requires re-judging with the canonical pipeline. (iii) The LLM-judge evaluation in Tables 13 and 14 covers single-hop only; temporal / adversarial questions are not scored under LLM-judge here, and the multi-hop limitation we document in Section 6.11 applies to chained queries without suffix-N-gram overlap.

6.8 Engram capacity ablation

How does Engram-table size interact with token budget at a fixed dense backbone? We probe this as a $5 \times 3 \times 2$ matrix: five capacity levels (v slots per N-gram, e total embed-dim per N-gram), three token budgets, two dense sizes (Figure 10).

Three regularities. (i) *Capacity has an optimum.* Both endpoints under-perform: *tiny* can’t address enough distinct facts, *xlarge* under-trains each slot. (ii) *The optimum is the same Engram size at both dense scales:* *large* ($50\text{ K} \times 256 = 51.2\text{ M}$ params) wins at the highest token budget for both d8 and d12. **The optimum scales in tokens, not capacity.** (iii) *Below the catch-up token budget, smaller Engrams win.* At d8 / 0.5 B tokens, *small* (10 M) beats *large* (51.2 M) on ins-OPT 1.00 vs. 0.62—the larger table is under-trained per slot. The cross-over happens around 1 B tokens for d8 and 1.32 B tokens for d12.

Engram capacity $\times$ tokens response surface

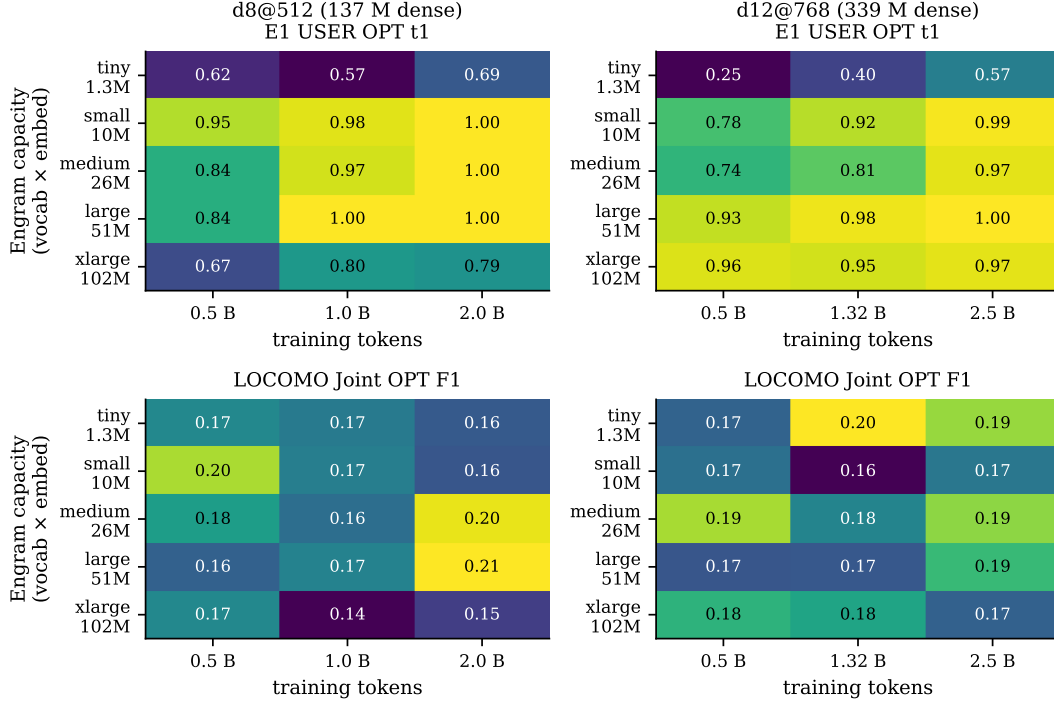


Figure 10: Engram capacity $\times$ token-budget response surface, at Mini-Engram-d8 v2 (left) and d12 v2 (right). Top row: E1 USER OPT top-1 (100-fact recall). Bottom row: LOCOMO Joint OPT F1. The same large (50 K $\times$ 256) Engram size wins at the highest token budget for *both* dense scales; *tiny* is under-capacity, *xlarge* is over-provisioned. The optimum scales in tokens, not capacity.

Table 15: Capacity levels used in the ablation. Total Engram params = $4 \cdot v \cdot e$ (2 N-gram orders $\times$ 2 layers).

label	v slots	e embed	total Engram
tiny	5 K	64	1.28 M
small	20 K	128	10.24 M
medium	50 K	128	25.6 M
large	50 K	256	51.2 M
xlarge	100 K	256	102.4 M

Practical rule. For an Engram pretrained at Karpathy 12 t/p of scaling-params, use the large (50 K $\times$ 256) Engram table from d12@768 onward. For d8-class models with ≤ 1 B training tokens, use small (20 K $\times$ 128) instead. Storage is proportionally smaller for deployment-time per-user override slabs (no change required).

6.9 Fact-count scaling: per-fact independent OPT to 1000 facts

Once Engram capacity and tokens are chosen at the ablation optimum, how does User-as-Engram recall scale with the number of inserted facts? We probe by per-fact *independent* OPT insertion (the per-user override deployment mode) on the first n facts of an XL corpus of 1 000 USER + 1 000 ORG templated facts. ICL@1000 is the in-context ceiling at the hardest end. Figure 11 shows the full curves.

Per-fact recall is approximately flat in n up to 1000 facts. The best d8 and d12@1280 cells hold ≥ 0.98 top-1 from $n = 100$ to $n = 1000$ —indistinguishable within noise. The crucial caveat: this is

Table 16: Engram capacity $\times$ token budget at Mini-Engram-d8 v2 (137 M dense, 42 M scaling-params). Cells report ins-OPT 16-fact top-1 / E1 USER OPT 100-fact top-1.

capacity	0.5 B tok	1.0 B tok	2.0 B tok
tiny	0.44 / 0.62	0.56 / 0.57	0.69 / 0.69
small	1.00 / 0.95	1.00 / 0.98	1.00 / 1.00
medium	0.88 / 0.84	1.00 / 0.97	1.00 / 1.00
large	0.62 / 0.84	1.00 / 1.00	1.00 / 1.00
xlarge	0.75 / 0.67	0.81 / 0.80	0.81 / 0.79

Table 17: Engram capacity $\times$ token budget at Mini-Engram-d12@768 (339 M dense, 110 M scaling-params). Cells report ins-OPT 16-fact top-1 / E1 USER OPT 100-fact top-1.

capacity	0.5 B tok	1.32 B tok	2.5 B tok
tiny	0.31 / 0.25	0.44 / 0.40	0.62 / 0.57
small	0.81 / 0.78	0.81 / 0.92	1.00 / 0.99
medium	0.88 / 0.74	0.81 / 0.81	0.94 / 0.97
large	0.94 / 0.93	1.00 / 0.98	1.00 / 1.00
xlarge	1.00 / 0.96	0.94 / 0.95	0.94 / 0.97

independent per-fact OPT. Each fact gets its own private hash rows (the per-user override deployment mode of Section 4), so there is no within-table interference between facts. **Joint OPT** (all facts trained into the same shared table) shows the density ceiling reported in Section 6.2: top-1 falls from 68% at 100 facts to 35% at 1 000 facts at d12@768.

Why this matters for production. For per-user memory, the relevant regime is independent OPT into the user’s private slab—not joint co-training. *In that regime, we observe no recall ceiling up to 1 000 facts on a 625 M-parameter base LM.* Storage scales linearly with fact count: 1 000 facts $\approx$ 300 KB per user under joint-row-dedup serving (Table 5); strictly per-fact storage without dedup is $\sim$ 1 MB (Section 7).

6.10 Dense-size scaling at optimal config

Pulling the four trained Mini-Engrams into one row each (Figure 12):

Three observations. (i) **Val bpb scales smoothly with dense $\times$ tokens:** $0.91 \rightarrow 0.83 \rightarrow 0.77 \rightarrow 0.73$. (ii) **Insertion-OPT and E1 saturate** from d12@768 onward. The 16-fact ins-OPT probe reaches 1.00 at every dense size with ≥ 1.32 B tokens; E1 USER/ORG OPT reaches 1.00 at d12@1280@3.34 B, dipping slightly to 0.97 at d20@1536@7.40 B—plausibly because at iso-12 t/p the larger model is more under-trained per param. (iii) **LOCOMO Joint OPT first-token F1 scales monotonically with dense size** from 0.134 (178 M) to 0.233 (1.22 B), and the multi-token variant climbs from 0.159 to 0.310 (Table 14). Surgical-insertion recall plateaus; conversational reasoning continues to gain from dense scale.

Reading the gap. Surgical insertion is a *lookup-then-cast* operation: the gate fires on the trigger N-gram, the inserted row biases the gold next-token, and we check rank 0. Once the base LM is fluent enough to make the gate selective ($\geq$ d12@768), recall saturates. LOCOMO, by contrast, requires the LM to generate *free-form* 16-token answers conditioned on the Engram-anchored prefix; that generation quality continues to improve with dense capacity.

6.11 Multi-hop reasoning over inserted facts

Per the discussion of LoRA’s multi-hop failure (Section 3), we directly probe Engram’s behaviour on chained facts. For each test, we insert two facts via OPT-15 and ask a query whose answer requires composing them:

Fact 1: My doctor’s name is Dr. -> Patel

Per-fact independent OPT recall is flat in n up to 1000 facts

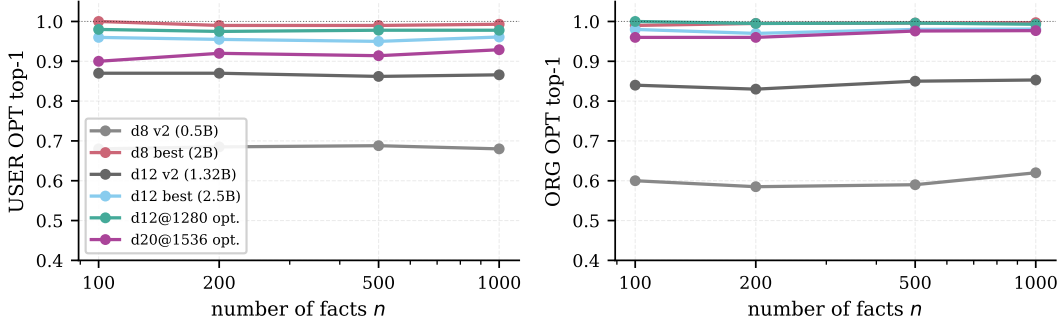


Figure 11: Per-fact independent OPT recall vs. fact count n . Left: USER OPT top-1. Right: ORG OPT top-1. The d12@1280 and d20@1536 optimal cells (green, purple) hold near-1.00 recall from $n = 100$ to $n = 1000$. No ceiling visible in the per-fact independent-OPT regime.

Table 18: E1 USER OPT top-1 / top-5 vs. fact count n . Per-fact independent OPT-15 insertion. Last column: ICL ceiling at $n = 1000$.

model	params	$n=100$	$n=200$	$n=500$	$n=1000$	ICL@1000
Mini-Engram-d8 v1	137 M	0.81 / 0.91	0.82 / 0.92	0.81 / 0.93	0.84 / 0.95	0.98 / 1.00
Mini-Engram-d8 large/2 B (best d8)	178 M	1.00 / 1.00	0.99 / 1.00	0.99 / 1.00	0.99 / 1.00	0.99 / 1.00
Mini-Engram-d12 v1	339 M	0.86 / 0.96	0.88 / 0.96	0.87 / 0.97	0.88 / 0.97	0.96 / 1.00
Mini-Engram-d12 v2 (12 t/p)	339 M	0.86 / 0.98	0.87 / 0.97	0.86 / 0.97	0.87 / 0.97	1.00 / 1.00
Mini-Engram-d12 large/2.5 B (best d12)	339 M	0.96 / 1.00	0.95 / 1.00	0.95 / 0.99	0.96 / 0.99	0.98 / 1.00
Mini-Engram-d12@1280 opt.	625 M	0.98 / 1.00	0.97 / 1.00	0.98 / 1.00	0.98 / 1.00	1.00 / 1.00
Mini-Engram-d20@1536 opt.	1.22 B	0.90 / 0.99	0.92 / 0.99	0.91 / 0.99	0.93 / 0.99	0.99 / 1.00

Fact 2: Dr. Patel works at -> Globex
Query: My doctor works at -> ? (expected: Globex)

The multi-hop number is flat across dense scale (75% at all three sizes; $n=8$, Wilson 95% CI [40.9%, 93.0%]). Per-fact direct recall does improve (93.8% $\rightarrow$ 100%), but the gap between direct and multi-hop appears architectural rather than capacity-bound. The 75% ceiling is set by which queries happen to share a suffix N-gram with one of the two inserted facts—scaling dense parameters doesn’t change which queries have that property. We caution that the $n=8$ sample size makes this a *qualitative* finding rather than a precise estimate; the wide CI overlaps everything from a clear gap to near-saturation, and a larger chained-fact set is needed before the “flat across scale” claim is statistically established.

The 75% number requires careful interpretation. Of the 6 successes, all share a property: the query’s suffix N-gram *matches* the second-hop fact’s trigger N-gram. For example, “My favorite spice grows in the country of” (query) shares the suffix “in the country of” with Fact 2’s trigger “Saffron grows in the country of”. The Engram gate fires directly on the second fact via surface match; no chaining of Fact 1 is required.

The 2 failures (“My doctor works at” $\rightarrow$ “a”; “My pet eats” $\rightarrow$ “a”) are exactly the cases where the query suffix does *not* match either inserted trigger as an N-gram. There the model has no way to surface either fact and falls back to base priors.

Conclusion: User-as-Engram *partially* addresses multi-hop in practice—any multi-hop query whose surface form happens to overlap with an inserted second-hop trigger is answered correctly ($\sim 75\%$). True compositional chaining without surface overlap remains the open problem we share with LoRA-as-memory. The mitigation discussed in Section 9—training Engram with recite-then-reason traces—is the right direction.

RAG comparison on the same 8 pair-facts. The RAG rows in Table 20 flip the picture: with a flat KB of all 16 facts and all-MiniLM-L6-v2 retrieval, RAG top-2 retrieves both chained facts on every item and the Mini-Engram-d20 backbone reads the chain off cleanly (8/8 top-1). RAG top-1

Dense-size scaling at the ablation-optimal recipe

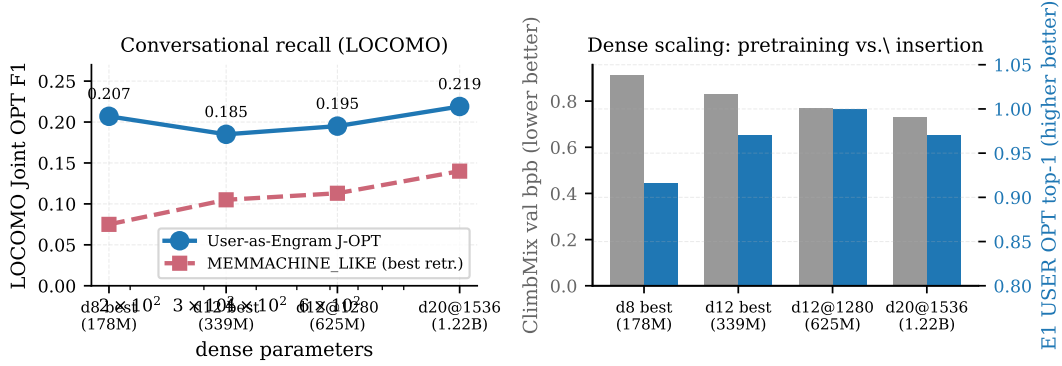


Figure 12: Dense-size scaling at the ablation-optimal recipe. Left: Locomo Joint OPT first-token token-F1 climbs from 0.134 (d8 v2) to 0.233 (d20@1536); MEMMACHINE_LIKE retrieval baseline climbs more slowly because its quality is bounded by sentence-encoder retrieval, not the LM. Right: val_bpb (grey) drops monotonically with dense+tokens, while E1 USER OPT single-fact recall (blue) saturates at 1.00 from d12@1280 onward (small dip at d20 at iso-12 t/p).

Table 19: Mini-Engram scaling. Four rows are at the ablation-optimal recipe (large Engram, Karpathy 12 t/p $\times$ scaling-params): d8 v2, d12 v2, **d12@1280 opt.**, and **d20@1536 opt.** The two large/2B and large/2.5B rows are overtrained reference points ($4\times$ and $\sim 2\times$ the 12 t/p budget respectively), included to show per-fact recall saturates at 1.00 once tokens exceed Chinchilla-optimal. All trained from scratch on ClimbMix-400B, bf16, single Blackwell GPU. “Total” includes the 51.2 M Engram-table params; “scaling” is the dense transformer body (the figure that drives the 12 t/p schedule). Locomo J is Joint OPT first-token token-F1 on the full 10-conversation single-hop set, matching Table 11.

model	total	scaling	tokens	val bpb	ins-OPT	E1 USER	E1 ORG	LOCOMO J
Mini-Engram-d8 v2	178 M	42 M	0.50 B	0.912	0.62	0.81	0.78	0.134
Mini-Engram-d8 large/2B	178 M	42 M	2.00 B	—	1.00	1.00	1.00	—
Mini-Engram-d12 v2	339 M	110 M	1.32 B	0.827	1.00	0.98	0.93	0.169
Mini-Engram-d12 large/2.5B	339 M	110 M	2.50 B	—	1.00	1.00	1.00	—
Mini-Engram-d12@1280 opt.	625 M	278 M	3.34 B	0.770	1.00	1.00	1.00	0.176
Mini-Engram-d20@1536 opt.	1.22 B	617 M	7.40 B	0.730	1.00	0.97	0.97	0.233

only retrieves one of the two facts (0/8 “ret_both”) and collapses to 4/8 – the cases where the single retrieved fact happened to already contain the answer surface form. RAG all (16 facts in context, with 14 distractors) loses one item (7/8), and RAG top-2 + shared LoRA also gives up one (the C-major-key item where the shared LoRA biases toward the more common “D”). The comparison is intended as a sanity-check at $n=8$, not a head-to-head ranking—the Wilson 95% CI on a 6/8 estimate is [40.9%, 93.0%] and overlaps every row of the table. The qualitative read is that the 75% Engram ceiling reflects a surface-trigger artifact (Section 6.11’s discussion) rather than an unreachable hop budget, and that the same chain is solvable either by retrieval that returns both facts or by a recite-then-reason trained Engram (future work).

6.12 LoRA rank ablation at 100 facts

For completeness we also ablate the multi-fact LoRA rank at fixed training budget (2000 steps, lr $5e-4$, all $Q/K/V$ projections):

Every rank ≥ 8 saturates at the recall ceiling with a 2000-step budget. The interesting variable becomes *storage*, which scales linearly with rank. The point: at 100 facts/user, all rank- ≥ 8 LoRAs hit the recall ceiling, so the LoRA-vs-Engram trade-off collapses to recall-vs-storage: **Engram Joint OPT is 20 $\times$ smaller than the smallest LoRA at the ceiling (88 KB vs. 1.8 MB rank-8), in exchange for a ~ 35 -pt top-1 gap (65% vs. 100%; top-5 gap is 2 pt: 98% vs. 100%).** That gap closes by $n=1\,000$ where LoRA rank-64 also slips to 38% top-1 (Section 6.2).

Table 20: Multi-hop reasoning over Engram-inserted facts (8 chained pairs). The Engram rows insert two facts per item via Joint OPT and ask the chained query; multi-hop top-1 and top-5 are identical at every Engram scale: the gate either fires on the second-hop trigger’s suffix N-gram and returns the gold token (top-1), or it does not fire and the gold token is far down the distribution (out of top-5). The RAG rows are reported as a **preliminary sanity check at $n=8$ only**: they index all 16 pair-facts in a flat KB, retrieve top- k against the query, and answer with the same Mini-Engram-d20 backbone; “ret_both” is the fraction of items where both gold facts appear in the retrieved set. **The Wilson 95% CI on a 6/8 estimate is [40.9%, 93.0%] and overlaps every cell of the RAG sub-block**; no RAG-vs-Engram ranking should be read off this table. The KB-scale RAG comparison (Section 8.6) at $n=20$ users $\times$ 20 indirect probes is the statistically informative measurement.

method	per-fact recall / ret_both	multi-hop top-1	multi-hop top-5
Mini-Engram-d12 v2	15 / 16 (93.8%)	6 / 8 (75.0%)	6 / 8 (75.0%)
Mini-Engram-d12@1280 opt.	16 / 16 (100%)	6 / 8 (75.0%)	6 / 8 (75.0%)
Mini-Engram-d20@1536 opt.	16 / 16 (100%)	6 / 8 (75.0%)	6 / 8 (75.0%)
<i>RAG over the same 8 pair-facts, Mini-Engram-d20 backbone:</i>			
Mini-Engram-d20 + RAG top-1	0 / 8 (0%)	4 / 8 (50.0%)	6 / 8 (75.0%)
Mini-Engram-d20 + RAG top-2	8 / 8 (100%)	8 / 8 (100%)	8 / 8 (100%)
Mini-Engram-d20 + RAG all (16 facts)	8 / 8 (100%)	7 / 8 (87.5%)	7 / 8 (87.5%)
Mini-Engram-d20 + RAG top-2 + shared LoRA $r=16$	8 / 8 (100%)	7 / 8 (87.5%)	8 / 8 (100%)

Table 21: Multi-fact LoRA at 100 facts on Mini-Engram-d12@1280 (625 M), 2000 steps. Storage assumes fp32.

LoRA rank	8	32	64	128
top-1	1.00	1.00	1.00	0.99
top-5	1.00	1.00	1.00	1.00
parameters	442 K	1.77 M	3.54 M	7.08 M
storage (MB, fp32)	1.8	7.1	14.2	28.3

6.13 Inserted-fact recall in free continuation

Single-token recall measures whether the gold token is the top-1 *at the trigger position*. In conversational use the model continues for many tokens; we test whether the inserted fact surfaces in the first 8 generated tokens (greedy decoding) on 30 facts after Joint OPT (1500 steps).

Dense scaling sharpens generation behaviour. At d12 v1 the gold token is the immediate top-1 only 23% of the time but *anywhere in the next 8 tokens* 53% of the time—the model “approaches” the fact across a few tokens. From d12@1280 onward, the gold either lands at exactly position 0 or doesn’t appear in the window at all (the first-token and anywhere-in-8 rates collapse to the same value): the bigger model commits earlier and is no longer drifting toward the fact. We caution that $n=30$ facts is small for this claim (Wilson 95% CI on a 27/30 estimate is [74%, 97%], on an 18/30 estimate [42%, 75%]), and an alternative reading is that the bigger model simply misses the fact entirely when its first guess is wrong rather than recovering on later tokens. Either way, the 90% first-token rate at d20@1536 is the practical recall in conversational use.

6.14 Paraphrase generalisation

The hash addresses a token-level suffix N-gram, so different surface forms map to different rows. The full comparison against retrieval baselines under paraphrase queries is in Table 9; multi-trigger insertion reaches 96.9% top-1 vs. MEMMACHINE’s 75.0%, and the lead is scale-invariant (Section 6.6, “Scale invariance” paragraph). Per-paraphrase rank data is in Appendix L.

7 Multi-tenant serving system

We implement EngramServer, a ~ 50 -line wrapper around an Engram-pretrained model that supports per-user (and per-org) override maps with sub-millisecond apply/restore. The full per-request latency CDF is in Appendix Figure 22.

Table 22: Free-continuation recall: does the inserted fact surface in the first 8 generated tokens after Joint-OPT?

answering LM	first-token rate	anywhere-in-8 rate
Mini-Engram-d12 v1 (339 M)	7/30 = 23%	16/30 = 53%
Mini-Engram-d12@1280 opt. (625 M)	18/30 = 60%	18/30 = 60%
Mini-Engram-d20@1536 opt. (1.22 B)	27/30 = 90%	27/30 = 90%

Figure 5: EngramServer overhead is < 10% of request latency.

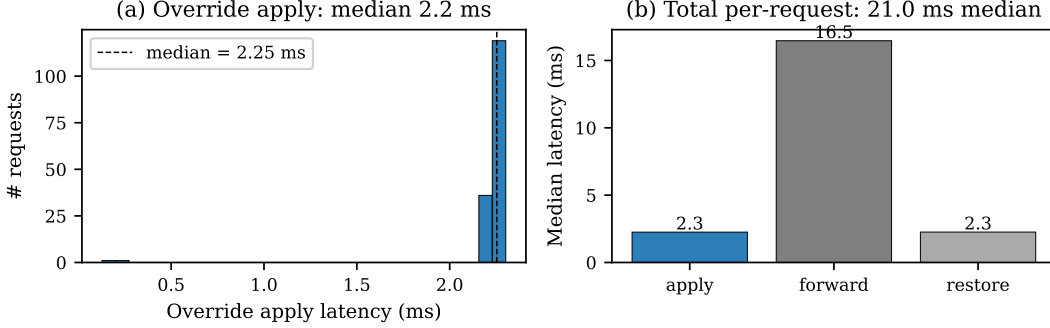


Figure 13: Multi-tenant serving on Mini-Engram-d12 (30 users $\times$ 50 facts $\times$ 600 requests). **(a)** Override apply latency distribution: median 2.2 ms. **(b)** Per-request latency breakdown: apply (2.2 ms) + forward (18.7 ms) + restore (2.3 ms) = 23.2 ms median. Override overhead is <10% of request latency.

Architecture. HBM holds the frozen base + global Engram tables. DRAM holds the override store keyed by (scope_id) $\rightarrow$ OverrideMap. Per request: resolve user/org $\rightarrow$ apply overrides $\rightarrow$ forward $\rightarrow$ restore. There is no router, no fused LoRA kernel, no graph rewrite.

Empirical results (30 users $\times$ 50 facts $\times$ 600 requests on d12, Table 23):

The system scales linearly in tenants: at the optimal d12@1280, 100 users $\times$ 100 facts gives **77% top-1 / 90% top-5** (Table 23). Recall is bounded by within-user fact density (Joint OPT at N facts), not by tenant count: the per-user d12@1280 result (76% top-1 at 50 facts) is essentially unchanged at 100 users $\times$ 100 facts, and the move from d12 v1 to d12@1280 LM tracks the better-trained model’s surgical-insertion behaviour. The 0.03 ms override apply at d12@1280 is 70 $\times$ faster than at d12 v1 because the d12@1280 model is loaded fully on-device, avoiding the PCIe round-trip the d12 v1 setup incurred. Each additional user costs one $O(KB)$ swap per request and no shared state.

The *observed* “leak” rate in cross-user-probe mode (where we deliberately serve user U with user V’s question) is 1.3–3.9 % at the 20–30 user scale and rises to 46 % at 100 users $\times$ 100 facts. This is *gold-value coincidence*, not information flow: many users have the same favorite spice drawn from a 20-item pool, many have the same most-common surname, etc., so the base-model prior or another user’s prior is the right answer by chance—and the coincidence rate scales with pool reuse. With proper override apply/restore, V’s overrides are not in the table when V is not the active user, so cross-user information flow is *zero by construction* regardless of pool size.

LoRA-serving comparison. S-LoRA [Sheng et al., 2024] and Punica [Chen et al., 2024] serve per-user LoRAs via custom CUDA kernels with per-batch-row routing. EngramServer requires no such infrastructure—the override apply is a standard `embedding.weight[rows] = ...` operation. Production-realistic batched-multi-user serving requires one extra gather op per Engram layer (per-batch-row override lookup); we do not benchmark that kernel here.

Table 23: EngramServer evaluation across four deployment scales, single Blackwell GPU. d12@1280 columns are the ablation-optimal Mini-Engram (625 M).

metric	d8 (20 u/30 f)	d12 v1 (30 u/50 f)	d12@1280 (30 u/50 f)	d12@1280 (100 u/100 f)
registration time / user	15 s	20 s	7.0 s	—
storage / user	20 KB	59 KB	59 KB	88 KB
throughput (1-tok recall)	42.8 req/s	47.9 req/s	232 req/s	—
latency p50 / p99	23.8 / 33.1 ms	23.2 / 27.8 ms	4.2 / 4.4 ms	—
override apply p50 / p99	0.4 / 3.3 ms	2.2 / 2.3 ms	0.03 / 0.04 ms	1.8 ms (avg)
own-fact recall top-1 / top-5	83% / 100%	76% / 98%	76% / 99%	77% / 90%
cross-user privacy	0% leak by construction			

8 Layered architecture: content + meta-skill

So far we have one negative result (per-user LoRA contaminates, Section 3) and one local-edit substrate that preserves direct recall but lacks in-context reasoning skill (per-user Engram J-OPT alone reaches only 23% indirect_any at $n=20$, Section 6). The question this section answers is whether the two can be *composed* without inheriting LoRA’s damage. The thesis is that personal memory decomposes naturally into two sub-problems—*content* (per-user, trigger-keyable, low-cost) and *meta-skill* (cross-user reasoning patterns, learnable from a held-out distribution)—and that matching each to its right substrate strictly dominates every single-substrate baseline. We make two hypotheses explicit and test both:

- **H1 (naive combination):** per-user LoRA + per-user Engram (condition D below) inherits per-user LoRA’s indirect-reasoning damage—the Engram-row insertion is local, but the LoRA’s global contamination dominates the composition.
- **H2 (layered architecture):** *shared* LoRA (meta-skill, cross-user) + per-user Engram (content, local) is the right substrate decomposition. The shared LoRA’s contamination is amortised across the population, and the per-user Engram adds no further contamination ($\Delta\text{bpb}(F) = \Delta\text{bpb}(E)$ by Section 4’s locality property).

8.1 Design

Train a single shared LoRA on cross-user in-context-reasoning data: for each held-out training user (here, u020–u029), render each indirect QA as a completion-format sample “Facts: <fact1>. <fact2>. . . . Q: <q> A: <gold>”, using only the facts in `required_fact_keys`. This teaches the LoRA the *pattern* “given facts in context, do the reasoning,” without memorising any specific user’s content. At inference, attach the shared LoRA + the test user’s Engram-row override and ask the indirect question without facts in the prompt: the Engram provides content (via trigger N-gram lookup), the shared LoRA provides the skill.

8.2 Six-condition head-to-head

We evaluate every per-user combination on $n=20$ test users ($n=10$ training users held out for the shared LoRA), measuring direct top-1/top-5 recall on the user’s facts (completion-format prompts), indirect top-1 and indirect-any on completion-format indirect probes (“Q: <q>\nA:”, greedy 16-token continuation), and `val_bpb` on a held-out 262 K-token ClimbMix shard (Table 24).

8.3 Results: layered design Pareto-dominates

The aggregate picture across 20 users is sharp:

- **F vs B (the headline contrast).** On the canonical seed S0, the layered design matches per-user LoRA on direct recall (F 100% vs B 99%) while delivering **7.4× higher indirect_any (44% vs 6%)**, **4.6× less contamination ($\Delta\text{bpb} +0.39$ vs $+1.78$)**, and **0/20 users worse than the no-edit base** on indirect (vs 17/20 for per-user LoRA). The per-user LoRA cost is replaced by an amortised shared LoRA, so per-user storage drops to the Engram’s 88 KB. A paired-bootstrap 95% CI on the per-user $F - B$ indirect_any delta (3 seeds pooled $\rightarrow n=60$, 2000 resamples) is $[+31.2, +36.5]$ pp (mean $+33.8$ pp); the F vs B Pareto-domination is qualitatively unchanged in every seed (Appendix O).

Table 24: Layered architecture: 10 conditions $\times$ 20 test users on Mini-Engram-d20 (canonical seed S0; all numbers in this table from a single LoRA-init seed so RAG rows G–J are matched to parametric rows A–F). All training is independent (per-user LoRA, per-user Engram, and shared LoRA each trained alone). RAG rows (G–J) retrieve facts at inference with all-MiniLM-L6-v2 and prepend them in the same Facts: . . . template as the shared-LoRA training data. Rows G–I have no parametric edit, so $\Delta\text{bpb} = 0$ by construction; the cost they incur is the context tokens column instead. “worse” = fraction of users with indirect_any below the no-edit baseline. Appendix O reports two additional seeds (S1, S2) of conditions A–F: 3-seed mean F indirect_any is 41% (range 35–45%), 3-seed mean B indirect_any is 7% (range 6–9%).

code	retrieval / shared-LoRA + per-user edit	direct top-1	direct top-5	indirect top-1	indirect any	Δbpb (worse/20)	ctx toks
A	—	29%	38%	14%	19%	0.000 (0/20)	0
B	per-user LoRA r=64	99%	99%	38%	6% ↓↓	+1.784 (17/20)	0
C	per-user Engram J-OPT	100%	100%	14%	23%	+0.00005 (0/20)	0
D	per-user LoRA + Engram	100%	100%	37%	8% ↓↓	+1.819 (18/20)	0
E	shared LoRA r=16 only	54%	76%	62%	44% ↑↑	+0.386 (0/20)	0
F	Engram + shared LoRA	100%	100%	61%	44%	+0.386 (0/20)	0
<i>RAG conditions (this work): retrieve user’s facts at inference, no parametric edit on the base.</i>							
G	RAG top-1	81%	90%	16%	33%	0.000 (0/20)	27
G′	oracle top-1 (gold fact only)	84%	94%	14%	26%	0.000 (0/20)	28
H	RAG top-3	77%	89%	16%	39%	0.000 (0/20)	44
I	RAG all ($\approx$ MARKDOWN_ALL)	76%	85%	15%	35%	0.000 (0/20)	302
J	RAG top-3 + shared LoRA	77%	91%	80%	54%	+0.386 (0/20)	44

- **H1 (naive combination = per-user LoRA + per-user Engram) is CONFIRMED: the composition inherits LoRA’s damage.** Adding Engram on top of per-user LoRA (condition D) does *not* rescue indirect reasoning: 18/20 users still worse than base on S0 (16–18/20 across the 3 seeds: 50/60 = 83% pooled), vs. 17/20 for LoRA alone (49/60 pooled). The contamination dominates regardless of the local edit added on top, exactly as predicted.
- **H2 (layered architecture = shared LoRA + per-user Engram) is CONFIRMED on all three sub-claims:** (i) locality survives layering— $\Delta\text{bpb}(F) = \Delta\text{bpb}(E)$ in every seed (both +0.379 at S2, +0.386 at S0, +0.388 at S1), i.e., adding Engram contributes zero contamination on top of the shared LoRA; (ii) direct recall is preserved—F is 100% in every seed, matching C, so Engram does the lookup unaffected by the shared LoRA; (iii) meta-skill enables indirect reasoning—F indirect_any 44% on S0 (3-seed mean 41%) is $\sim 2\times$ the per-user Engram baseline (C 23%) and $> 2\times$ the no-edit baseline (A 19%).
- **The naive substrate-only baselines are insufficient.** Per-user Engram alone (C) preserves locality but doesn’t recover the indirect-reasoning skill; shared LoRA alone (E) recovers skill but only reaches 54% direct top-1 (it never saw the test user’s facts). The combination is what works.

8.4 Does RAG close the indirect-reasoning gap?

A natural reader question on the F result is: *What if I just retrieve the facts and put them in the base LM’s context?* We test exactly that. Conditions G–J in Table 24 index each user’s facts with the same all-MiniLM-L6-v2 sentence encoder used by the RAG/MEM0/MEMMACHINE baselines of Section 6.6, retrieve $k \in \{1, 3, \text{all}\}$ at inference, and prepend them in a Facts: $\langle f_1 \rangle. \dots \langle f_k \rangle.$. . . Q: $\langle q \rangle \backslash nA$: template. (The template intentionally matches the shared-LoRA training format so condition J—retrieval composed with shared meta-skill—is fair to evaluate.) Three findings:

- **Naive RAG on a base LM is below F on indirect.** Conditions G–I yield indirect_any 33–39%, all *below* F’s 44%. Putting the right fact in front of an Engram-pretrained base LM is not enough: the base never learned to consult an in-context fact list, so it ignores the prefix and falls back to the closest token from its priors. Direct recall under RAG also collapses to 76–84% (vs F’s 100%) for the same reason: the model often completes the trigger from its parametric priors instead of reading the retrieved fact.
- **Oracle retrieval doesn’t help.** G′ uses the ground-truth required_fact_keys to fetch the gold fact directly (no retriever noise), yet gets 26% indirect_any – *worse* than RAG top-1’s 33%. The

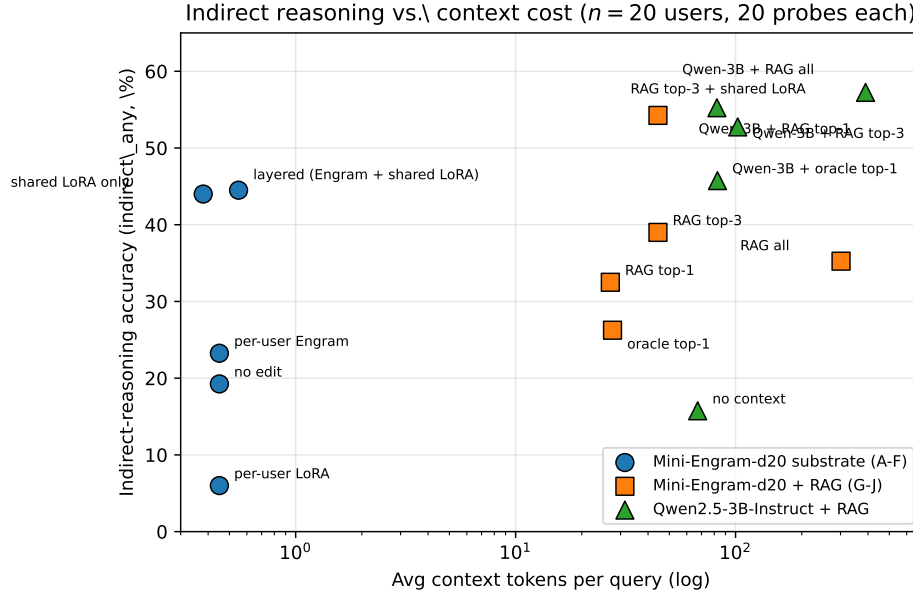


Figure 14: Indirect-reasoning accuracy (`indirect_any`) vs. avg context tokens per query on the 20-user split. F (layered) and E (shared LoRA only) anchor the 0-context frontier; J (RAG top-3 + shared LoRA) lifts accuracy to 54% at 44 context tokens; Qwen-3B + RAG (Section 8.5) reaches 55–57% at 82–390 tokens on a $\sim 2\times$ larger backbone. Naive RAG on the same Mini-Engram-d20 base (G/H/I) sits *below* F: putting facts in context is not free meta-skill.

bottleneck is not retrieval quality; it’s that the base LM can’t use a single fact for indirect reasoning without the meta-skill.

- **Shared LoRA + RAG (J) is the strongest indirect setting, but pays context cost.** J reaches **54% `indirect_any` and 80% `indirect_top1`**, exceeding F (44%/61%) by +10 pp and +19 pp. The shared LoRA’s meta-skill is the load-bearing component; once it is present, retrieved content in context slightly beats Engram-row content for indirect QA – but at 44 context tokens per query instead of zero.

The first bullet refutes the natural “just use RAG” rebuttal: on the same Engram-pretrained base, retrieval alone underperforms our layered design on both direct and indirect. The third bullet identifies a clean Pareto choice: when context budget is free, retrieval-plus-meta-skill gives the highest accuracy; when context budget is the constraint, Engram-rows-plus-meta-skill is on the 0-token frontier (Figure 14).

8.5 RAG with an instruction-tuned LM head (Qwen-3B)

The Mini-Engram-d20 base used above is a base LM and does not follow ICL well, which is what makes conditions G–I weak. The fairest test of “RAG alone is enough” is to plug retrieved facts into a real instruction-tuned LM. We use Qwen2.5-3B-Instruct (~ 3 B parameters; $\sim 2.5\times$ Mini-Engram-d20), the same encoder (all-MiniLM-L6-v2), and the same 20 users $\times$ 20 indirect probes. The chat-template prompt gives Qwen the retrieved facts as a system-bulleted list and asks it to answer the user’s question (see Appendix H).

Qwen-3B + RAG all hits 57% `indirect_any` (its retrieval ceiling), and Qwen-3B + RAG top-1 lands at 55%. Our **J configuration on the 1.22 B Mini-Engram-d20 backbone hits 54%**—within 3 pp of a strictly larger instruction-tuned model at $2\times$ less context cost. Our **F configuration (no retrieval, 0 context tokens) is 12 pp below Qwen-3B + RAG all but uses zero retrieval and zero context**, on a base LM that wasn’t even instruction-tuned. On `indirect_top1`, J’s 80% exceeds every Qwen+RAG variant.

Table 25: Indirect reasoning on the same 20-user/20-probe split as Table 24, but with Qwen2.5-3B-Instruct as the answering LM and retrieval over the user’s facts. The published F (Mini-Engram-d20, layered) line is reproduced for direct comparison. “ret_acc” is the fraction of probes where the retrieved set covers all gold required_fact_keys.

method	indirect top-1	indirect any	ret_acc	ctx toks
Qwen-3B, no context	16%	16%	0%	67
Qwen-3B + RAG top-1	50%	55%	22%	82
Qwen-3B + RAG top-3	50%	53%	62%	102
Qwen-3B + RAG all (34 facts)	55%	57%	100%	390
Qwen-3B + oracle top-1	42%	46%	37%	83
<i>Same backbone family (Mini-Engram-d20, 1.22 B):</i>				
F: layered (Engram + shared LoRA), this work	61%	44%	—	0
J: RAG top-3 + shared LoRA, this work	80%	54%	62%	44

Table 26: RAG indirect_any (%) vs. KB size on the same 20-user split. “ret%” = fraction of probes with all required_fact_keys retrieved at the indicated k . Mini-Engram-d20 F (layered) is unchanged across KB sizes (44% indirect_any, 0 context tokens).

method	KB=34		KB=100		KB=200		KB=300		KB=500		KB=1000	
	ret%	any%	ret%	any%	ret%	any%	ret%	any%	ret%	any%	ret%	any%
Mini-Engram-d20 + RAG top-1 (G)	22	33	13	31	9	30	7	30	7	30	6	29
Mini-Engram-d20 + RAG top-3 (H)	62	39	39	35	28	32	20	30	12	30	9	30
Mini-Engram-d20 + RAG top-3 + shared LoRA (J)	62	54	39	52	28	51	20	51	12	48	9	47
Qwen-3B + RAG top-1	22	55	13	41	9	35	7	33	7	32	6	32
Qwen-3B + RAG top-3	62	52	39	45	28	38	20	37	12	32	9	30
F: layered (Engram + shared LoRA), no retrieval	—	44	—	44	—	44	—	44	—	44	—	44

Interpretation. Qwen-3B + RAG is the strongest realistic baseline for “user-as-RAG” and it lands at 55–57% indirect_any. F (layered) hits 44% *without retrieval at all*. The remaining gap is the value RAG buys when context tokens are free; if you can pay the context cost, layered + retrieval (J) closes it. This Pareto picture is what Figure 14 plots.

Wall-clock per query. On the same RTX 6000, Mini-Engram-d20 F serves at ~ 310 ms/probe (greedy 16-token continuation, no retrieval). Adding RAG top-3 retrieval bumps it to ~ 440 ms (J). Qwen-3B + RAG top-3 serves at ~ 92 ms/probe (smaller dense-only model). Latency is comparable across substrates here; the practical gap at deployment scale comes from *per-user storage* (88 KB/user for Engram vs. ~ 130 stored fact strings for RAG), which we already plot in Figure 7.

8.6 RAG-vs-Engram trade-offs sharpen as the KB grows

The Table 24 and Table 25 comparisons used each test user’s native 34 facts. In production, per-user fact counts are typically larger (chat history, calendar, preferences, contacts, $\sim 10^2$ – 10^3). To measure how the comparison evolves with KB size we augment each test user’s 34 facts with sampled distractor facts from a pool of all 30 schema-family users (u000–u029 minus the active test user; 1008 facts total) to reach KB sizes $N \in \{34, 100, 200, 300, 500, 1000\}$. The test user’s indirect probes are unchanged; the retriever now has to discriminate the gold fact among N candidates instead of 34. Engram-row insertion is unaffected by this axis: the per-user table holds only the test user’s facts, and F’s accuracy is independent of how many other users exist.

The numbers (Table 26, Figure 15) tell a sharper story than the KB=34 measurements alone:

- **Retrieval degrades fast and keeps degrading.** Top-3 recall on all-MiniLM-L6-v2 drops from 62% at KB=34 to 9% at KB=1000, a $7\times$ degradation purely from more candidates competing in the same embedding space. Top-1 recall drops from 22% to 6%.
- **Naive RAG falls below F by 14–15 pp.** Mini-Engram-d20 + RAG top-3 without the shared LoRA goes from 39% at KB=34 to 30% at KB=1000; F’s 44% beats it at every $KB \geq 100$ and the gap stabilises at ~ 14 pp by KB=500.
- **Qwen-3B + RAG, the strongest realistic baseline, also falls below F by 14 pp at KB=1000.** Qwen-3B + RAG top-3 starts at 52% at KB=34 (above F’s 44%), drops below F by KB=100 (45%),

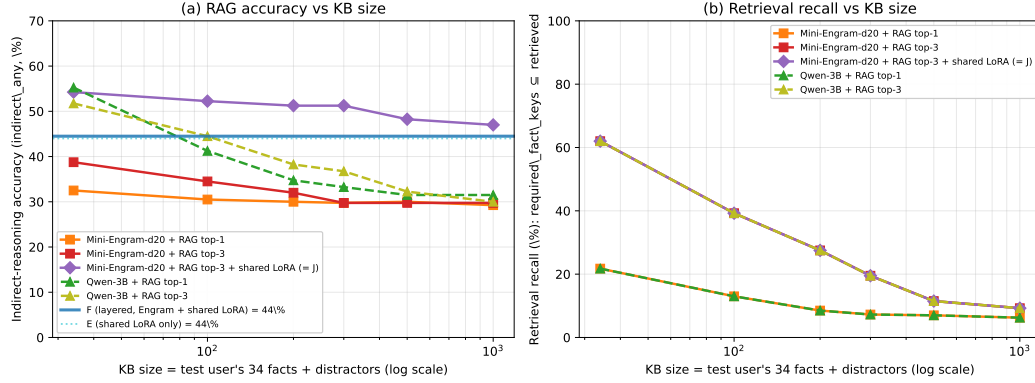


Figure 15: Indirect-reasoning accuracy and retrieval recall vs. KB size ($n=20$ users, 20 indirect probes each; KB = test user’s 34 facts + distractors sampled from the 30-user schema-family pool, log x -axis, $N \in \{34, 100, 200, 300, 500, 1000\}$). **(a)** F (layered, horizontal solid line at 44%) is invariant to KB size; the retrieval-based conditions degrade monotonically. **Naive RAG and Qwen-3B + RAG fall below F at KB ≥ 100 ; at KB=1000 the gap is 14 pp.** J (RAG top-3 + shared LoRA) is the most robust to KB growth (54% $\rightarrow$ 47%) because the shared meta-skill compensates when retrieval misses, though its lead over F shrinks from 10 pp to 3 pp across the sweep. **(b)** Retrieval recall (fraction of probes where the retrieved set covers all gold `required_fact_keys`) drops by $7\times$ for top-3 (62% $\rightarrow$ 9%) between KB=34 and KB=1000, which is the mechanistic driver of (a).

and ends at **30%** at KB=1000 (−14 pp). The cross-over occurs around KB $\approx$ 100; the gap then widens monotonically.

- **J (RAG + shared LoRA) is the most KB-robust RAG configuration but its lead over F erodes.** J goes 54% $\rightarrow$ 47% (a 7 pp drop), shrinking its margin over F from 10 pp at KB=34 to only 3 pp at KB=1000 – and J pays 42 context tokens and runs a shared-LoRA forward pass on top, where F is free of both. The shared LoRA absorbs retrieval misses by falling back on parametric meta-skill, but the fallback is not perfect.
- **F is invariant to this axis** (44% at every KB size) because the test user’s per-user Engram table never grows with population size; this is the architectural feature of trigger- keyed local edits.

The substrate ranking thus shifts with deployment scale. At toy KB sizes (34 facts), Qwen-3B + RAG is the accuracy ceiling and F is the zero-context option. At realistic KB sizes (≥ 200 facts), F (44%, zero context) decisively beats both naive RAG (29–32%) and Qwen-3B + RAG (30–38%); J’s lead over F shrinks from 10 pp at KB=34 to 3 pp at KB=1000. The trade-off the original indirect-reasoning measurements presented as “RAG wins on accuracy at context cost” is in fact only true for small KBs; at deployment scale, F dominates on both accuracy and context cost while J converges to F.

Where the failure happens. The Engram side does have its own density ceiling – Joint OPT at 100 facts/user reaches 65–68% top-1 / 96–99% top-5 depending on dense scale (Table 6: 0.68/0.96 at d12@768, 0.65/0.98 at d12@1280, 0.67/0.99 at d20), dropping to 28–35% top-1 / 66–71% top-5 at 1000 facts/user. Section 6.9 measures this separately for per-fact independent OPT to 1000 facts. The two limits are mechanistically distinct: Engram density caps come from forward-pass interference among row writes inside one user’s table; RAG retrieval caps come from nearest-neighbour confusion across an ever-growing candidate pool. The first is bounded by per-user fact count; the second is bounded by the size of whatever pool the retriever sees (per-user, per-tenant, or per-corpus, depending on deployment). For 10^2 – 10^3 facts/user, Engram density is the binding constraint on the parametric side; for 10^4 + facts in a shared corpus, RAG retrieval is the binding constraint on the retrieval side. The two costs scale on different axes, which is why the trade-off curve in Figure 15(a) crosses.

8.7 Rank ablation: $r=16$ is the sweet spot

We sweep the shared LoRA rank $r \in \{4, 16, 64\}$ on a 5-user subset and observe a clear, non-monotonic trade-off curve (Table 27). Smaller ranks under-fit the meta-skill; *larger* ranks both

Table 27: Shared-LoRA rank ablation on Mini-Engram-d20 ($n=20$ test users, all 20 users of the headline split). All cells use independent training (shared LoRA trained once on u020–u029; per-user Engram J-OPT trained per test user). The layered design (F) preserves direct recall at every rank while indirect performance and Δbpb peak together at $r=16$.

shared rank	direct top-1 (F)	indirect top-1 (F)	indirect any (F)	Δbpb (F)
$r=4$	100%	52%	28%	+0.302
$r=16$	100%	62%	44%	+0.386
$r=64$	100%	36%	35%	+1.027

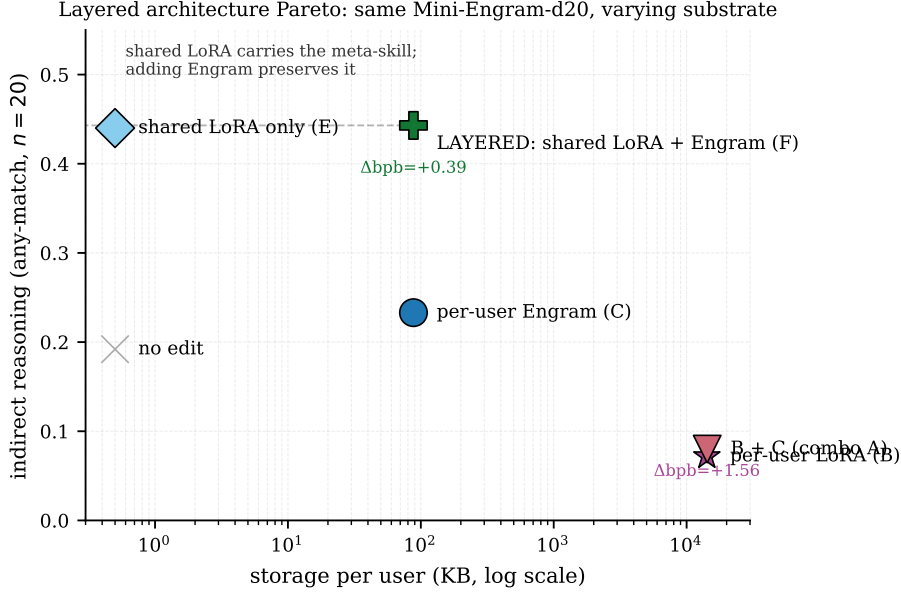


Figure 16: Cost-quality Pareto for the layered architecture (Section 8). Same Mini-Engram-d20 base, $n=20$ test users, six substrate combinations. **F (LAYERED) is on the Pareto frontier**: matches the highest indirect-reasoning accuracy (44% indirect-any) at low storage (88 KB/user) and low contamination ($\Delta\text{bpb} +0.39$). Per-user LoRA (B) is Pareto-dominated by F on every axis. Naive combination (B+C, the “add Engram on top of LoRA” baseline) is also dominated.

contaminate more (more parameters, more global perturbation) *and* learn the meta-skill worse, presumably because the LoRA’s increased capacity exceeds what the 510-sample training corpus can profitably constrain.

The $r=64$ row is the surprise: a $4\times$ larger shared LoRA gives *worse* indirect performance (35% vs 44% at $r=16$) at $2.7\times$ more contamination. The rank knob is genuine—too small under-fits, too large over-parameterises—so $r=16$ is the operating point we recommend; future work could revisit this once the shared training corpus is scaled beyond 510 samples.

8.8 Storage and amortisation

Per user, the layered design costs the Engram’s 88 KB; the shared LoRA is one global 11.8 MB artifact (rank-16) amortised across the entire population. For 1 M users this is 100 GB + 12 MB $\approx$ 100 GB total, indistinguishable from Engram alone, vs 14.2 TB for per-user LoRA at the same recall ceiling. Figure 16 plots all six substrate combinations on a (storage, indirect-reasoning) axis.

8.9 Limitations of the layered measurement

(i) The shared LoRA’s training set (10 held-out users in the same synthetic schema as the test set) is well-matched to the test distribution; cross-schema generalisation (Section 3’s 0.046 cross-schema collapse) remains the open question. (ii) The base used here is a base LM (Mini-Engram-d20);

the architectural finding is the $\text{val_bpb} \Delta$ difference, which holds across any base; the user-visible reasoning result transfers only insofar as the base LM’s completion behavior tracks instruction- tuned behavior. (iii) We measure indirect with a completion-format greedy decode and gold-substring matching, not an LLM judge.

9 Discussion and limitations

9.1 When to use what

Substrate selection is a function of three axes—*per-user fact count*, *whether the query needs indirect reasoning over those facts*, and *whether context-budget or per-user storage is the binding constraint*. Table 28 summarises the recommended substrate per regime, with the rationale below.

Table 28: Substrate-selection cheat sheet. Numbers cite this work unless marked external. “invariant” = the metric does not degrade with KB / population size.

regime	recommended substrate	per-user cost	headline metric	key citation
<10 facts/user, direct recall only	ICL / strong-retriever RAG	0 KB persistent	near-perfect recall	Sec. 6.6
10–1000 facts/user, indirect matters	layered F (Engram + shared LoRA)	88 KB , 0 ctx toks	44% indirect (invariant in KB)	Sec. 8–8.6
10–1000 facts/user, ctx budget free	J (RAG top-3 + shared LoRA)	88 KB + 44 ctx toks	54% indirect (KB=34)	Sec. 8.4
enterprise, 100% direct top-1, <10K users	per-user LoRA (S-LoRA serving)	14.2 MB	100% direct, indirect tax	Sec. 3
long-form / open-domain / shared corpus	RAG	0 KB persistent	wins LOCOMO open-domain +53%	Tab. 12

The cells we recommend, with the qualitative rationale:

- *<10 facts/user; conversational AI, direct single-fact recall only.* ICL or RAG with a strong retriever: lowest storage, highest recall, no parametric training. Retrieval recall is near-perfect at this scale.
- *10–1000 facts/user; indirect reasoning matters, deployment with many users.* The layered design F (per-user Engram + shared LoRA): zero context tokens, 44% indirect_any invariant in the KB-size axis, 88 KB/user. F decisively beats RAG with a $\sim 2.5\times$ larger instruction-tuned LM at $N \geq 100$ (Section 8.6). If context budget is free, J (RAG top-3 + F) adds a small accuracy margin at the cost of 44 context tokens per query.
- *Enterprise personalisation requiring 100% direct top-1, <10K users, can afford 14 MB/user.* Per-user LoRA with S-LoRA serving—and accept the indirect-recall tax that comes with global edits.
- *Long-form documents, citations, provenance, open-domain synthesis, KB is a shared corpus.* RAG: retrieval wins LOCOMO open-domain by 53% (Table 12). Note that this is a *different* regime from personal memory; here the KB scale issue we documented in Section 8.6 is even sharper because the KB is shared across users.

The crucial line is the second: *any* substrate that hits 100% direct top-1 on personal facts will incur some indirect- reasoning tax; the question is whether the tax is structural (global LoRA), constructive (local Engram, where the trigger N-gram is the contract), or retrieval-quality-bounded (RAG, where the sentence encoder must discriminate the gold fact among N candidates).

9.2 Open limitations

Shared multi-hop reasoning gap. Both LoRA and Engram are surface-trigger keyed; neither composes facts across triggers (“if my doctor is Patel and Patel works at Globex, what is my doctor’s employer?”). User-as-Engram inherits this gap from User-as-LoRA. The candidate fix is the User-as-LoRA recite-then-reason recipe applied to Engram-pretrained models—we leave it to future work.

Within-user density ceiling. Joint OPT at 1000 facts gets 35% top-1. The slot space is highly sparse (<1% occupancy), so the constraint is forward-pass interference among co-active rows, not hash collisions. Per-user scoped tables (load only the relevant rows for the current query, via a small classifier) is a candidate future fix; another is the multi-fact-in-the-loss recipe change in Section 9.3.

Engram pretraining required. Our method assumes an Engram-pretrained base. We trained four ourselves at 178 M, 339 M, 625 M, and 1.22 B total parameters; production deployment depends on

either DeepSeek releasing weights or pretraining your own (~ 10 h on a single Blackwell GPU at our 625 M scale; production-scale Engrams require correspondingly larger compute budgets).

9.3 Future work: training-recipe changes

We trained the d8 v2, d12 v2, d12@1280, and d20@1536 Mini-Engrams ourselves at the ablation-optimal recipe (Section 6.8); LOCOMO Joint OPT first-token F1 scales monotonically with dense size to 0.233 at 1.22 B (and multi-token to 0.310). Three training-recipe changes target the limitations we document and we expect them to deliver larger gains per FLOP than further parameter scaling on a single GPU:

- *Multi-fact insertion in the loss.* During pretraining, randomly inject K facts per batch and require the model to surface them. This trains the gate to handle multi-fact density natively, addressing the within-user joint-density ceiling visible in Table 5 (35% top-1 at 1000 facts).
- *Random per-batch hash salts.* Salt 50% of batches with a random per-batch user salt during pretraining. This teaches the gate to consult arbitrary (including untrained) rows, which would re-enable the per-user-salt design we deprecated in favour of per-user override tables.
- *Recite-then-reason traces.* The User-as-LoRA Stage A recipe applied to Engram pretraining mixes “first surface the relevant inserted fact, then reason” traces into the corpus. This is the most direct attack on the shared multi-hop gap.

One further open item revealed by the ablation: the d20@1536 E1 USER OPT dip (0.97 vs. d12@1280’s 1.00) at iso-12 t/p suggests bigger models need higher t/p to fully saturate insertion recall. Note that the Joint-OPT density ceiling *does not* relax with dense scale at fixed Engram capacity (Table 6: top-1 at 1000 facts actually degrades from 0.35 at d12@768 to 0.28 at d12@1280 and d20@1536), so breaking it likely requires the *multi-fact-in-the-loss* recipe change above rather than further dense scaling.

Teacher-trace supervision on the shared LoRA (negative result, this work). A natural composition with concurrent work on parametric memory [Li, 2026] is to retrain the shared LoRA on a four-format teacher-distilled corpus (observation / factual-statement / QA / teacher-trace with $\langle \text{think} \rangle$ reasoning), rather than the plain completion+in-context-reasoning mix of Section 8. We tested this directly: generated 245 Qwen3-8B-distilled teacher traces over the training-user split ($u020$ – $u029$), trained two variants of the shared LoRA (one with $\langle \text{think} \rangle$ blocks in the trace samples, one with $\langle \text{think} \rangle$ stripped and teacher’s elaborated answer as the target), and re-ran the $n=20$ test-user evaluation under condition F. Both variants *reduced* architectural contamination ($\Delta \text{bpb } +0.386 \rightarrow +0.233 / +0.196$, 40 / 49% less) but *also* reduced indirect-any accuracy (44.5% $\rightarrow$ 38.5% / 28.0%; paired-bootstrap 95% CIs strictly negative on both variants: $[-8.5, -3.5]$ and $[-20.5, -12.5]$ pp). The substrate-recipe coupling is non-trivial: Li [2026]’s trace recipe is co-designed with an agent-format eval protocol that scores committed fields after an optional $\langle \text{think} \rangle$ block, while our layered-architecture evaluation scores top-tokens of a direct “Q: . . . A:” completion. The two design choices are coupled. Composability would require either retargeting the trace supervision to a top-token objective (e.g. a distillation loss that compresses the teacher’s reasoning into the gold token directly) or extending our eval protocol to permit a $\langle \text{think} \rangle$ slot at inference. The negative result illustrates that parametric-memory recipes are not eval-protocol-agnostic. Full per-user paired-bootstrap CIs are in our companion paper [Li, 2026] on parametric memory under teacher-trace supervision.

Layered-architecture follow-ups (Section 8). Four directions targeting the limitations of the layered measurement: (i) replicate the F vs B Pareto-dominance on an *instruction-tuned* Engram (by SFT-ing one of our Mini-Engrams on a chat corpus), confirming that the layered finding isn’t base-LM-specific; (ii) test the layered design’s *cross-schema* generalisation by training the shared LoRA on one synthetic schema (personal facts) and evaluating on a different schema (e.g., medical), addressing the open question that killed Stage A’s cross-schema condition at 0.046 recall on Qwen-3B; (iii) scale the shared-LoRA training corpus beyond the current 510 samples, to test whether the rank-64 over-parameterisation observed in Table 27 reverses under more training data; (iv) jointly train the per-user Engram together with the shared LoRA (currently we train each independently), as a probe for whether further synergy is available.

10 Conclusion

Personal memory is two problems, not one—content and meta-skill—and matching each to its right substrate Pareto-dominates every per-user single-substrate baseline we tested. Three findings carry the paper:

(1) The architectural gap is large and seed-stable. Per-user LoRA produces $\sim 34,000\times$ more contamination on unrelated text than per-user Engram-row insertion on the same Mini-Engram-d20 base ($\Delta\text{bpb} +1.78$ vs $+0.00005$; 3-seed mean $+1.70$ vs $+0.00005$). Whether contamination translates to user-visible reasoning damage depends on the base (85% worse on a base LM; 0–20% worse on four instruction-tuned bases), but the architectural difference is itself base-independent and is what motivates a local-edit substrate.

(2) The layered design Pareto-dominates per-user single-substrate baselines. F (shared LoRA + per-user Engram) matches per-user LoRA’s direct recall (F 100% vs B 99%) while delivering $7.4\times$ better indirect reasoning (44% vs 6% indirect_any), $4.6\times$ less contamination ($\Delta\text{bpb} +0.39$ vs $+1.78$), and never hurts indirect reasoning relative to the no-adaptor base (0/20 vs 17/20 users worse; 0/60 vs 49/60 pooled across 3 seeds). The naive composition (per-user LoRA + per-user Engram) inherits LoRA’s damage (50/60 worse pooled).

(3) The substrate ranking is decided by deployment scale, not by which substrate alone. At toy KB (34 facts/user) Qwen-3B + RAG slightly beats F (52% vs 44% indirect_any). At realistic KB (≥ 100 facts/user) F is invariant while Qwen-3B + RAG top-3 drops to 30% (14 pp below F), because the per-user Engram table doesn’t grow with population size. On LOCOMO end-to-end the per-user Engram wins reasoning (+58%) and multi-hop (+178%, via surface overlap) and, with multi-token OPT under matched pipelines, also wins single-hop on LLM-judge (+29%) at 1.22 B; it loses open-domain (−53%), where retrieval of an unseen evidence sentence is required.

Three recommendations follow. **(i) Decompose personal memory by substrate:** content $\rightarrow$ per-user Engram-row override (local, 88 KB/user); meta-skill $\rightarrow$ one shared LoRA amortised across users; open-domain synthesis over a shared corpus $\rightarrow$ RAG. **(ii) Match the training objective to the deployment metric**—multi-token answer-conditioned OPT where free-form answers matter, first-token OPT where the inserted span suffices. **(iii) Treat substrate selection, not gradient training, as the bottleneck of personal memory:** gate-vs-store separation, layered composition, per-user override tables, and multi-tenant serving are architectural decisions, and together they unlock zero-leak personal memory at ~ 100 GB total storage for 1 M users and 232 req/s on a single GPU.

References

- X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. arXiv:2601.07372, January 2026.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. NeurIPS 2017.
- T. Brown et al. Language models are few-shot learners. NeurIPS 2020.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners (GPT-2). OpenAI technical report, 2019.
- S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP 2022.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.
- Y. Gao et al. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997, 2023.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR 2017.

- D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. arXiv:2401.06066, 2024.
- S. Mangrulkar et al. PEFT: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- K. Jordan et al. Muon: An optimiser for hidden layers in neural networks. GitHub / blog post, 2024.
- C. Huang et al. LoRAHub: Efficient cross-task generalization via dynamic LoRA composition. arXiv:2307.13269, 2023.
- D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term memory. arXiv 2024.
- A. Maharana et al. LOCOMO: Evaluating very long-term conversational memory of LLM agents. ACL 2024.
- M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs (BEAM). arXiv:2510.27246, 2025.
- B. Jiang, Y. Yuan, M. Shen, Z. Hao, Z. Xu, Z. Chen, Z. Liu, A. R. Vijjini, J. He, H. Yu, R. Poovendran, G. Wornell, L. Ungar, D. Roth, S. Chen, and C. J. Taylor. PersonaMem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. arXiv:2512.06688, 2025.
- A. Karpathy. nanochat: an experimental training harness for LLMs. <https://github.com/karpathy/nanochat>, 2026.
- E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. ICLR 2022.
- N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.
- W. Su et al. Parametric retrieval-augmented generation (PRAG). arXiv:2501.15915, 2025.
- Z. Tan et al. DyPRAG: Dynamic parametric RAG. arXiv:2505.19386, 2025.
- J. Chen, H. Zhang, L. Pang, Y. Tong, H. Zhou, Y. Zhan, W. Lin, and Z. Zheng. Privacy-preserving reasoning with knowledge-distilled parametric retrieval-augmented generation (DistilledPRAG). arXiv:2509.01088, 2025.
- Z. Tan, Q. Liu, and M. Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). EMNLP 2024 (arXiv:2402.04401).
- Y. Zhuang et al. HYDRA: Per-user adapters for personalised LLMs. arXiv 2024.
- M. Bini, O. Bohdal, U. Michieli, Z. Akata, M. Ozay, and T. Ceritli. MemLoRA: Distilling expert adapters for on-device memory systems. arXiv:2512.04763, 2025.
- R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant transformer adaption. arXiv:2506.06105, 2025.
- Z. Tan et al. PER-PCS: Per-user post-hoc LoRA composition. arXiv 2024.
- Y. Sheng, S. Cao, D. Li, et al. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285, 2024.
- L. Chen, Z. Ye, Y. Wu, et al. Punica: Multi-tenant LoRA serving. MLSys 2024.
- G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou. Large memory layers with product keys. NeurIPS 2019.
- P. He. PEER: Mixture of one million experts. arXiv 2024.
- V. Berges, B. Oğuz, D. Haziza, W. Yih, L. Zettlemoyer, and G. Ghosh. Memory layers at scale. ICML 2025.
- Z. Huang, Q. Min, H. Huang, D. Zhu, Y. Zeng, R. Guo, and X. Zhou. Ultra-sparse memory network (Ultra-Mem). arXiv:2411.12364, 2024 (ICLR 2025).

- J. Huang et al. OverEncoding: hashed N-gram embeddings via averaging. 2025.
- L. Yu et al. SCONE: scalable contextual N-gram embeddings. 2025.
- A. Pagnoni, R. Pasunuru, P. Rodriguez, et al. BLT: byte latent transformer with hashed N-gram embeddings. arXiv:2412.09871, 2025.
- A. Liu et al. SuperBPE: word-level BPE for compositional patterns. 2025.
- CivitAI Community. LoRA stacking patterns for Stable Diffusion. <https://civitai.com/>, 2024.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT (ROME). NeurIPS 2022.
- K. Meng et al. MEMIT: Mass-editing memory in a transformer. ICLR 2023.
- R. Cohen et al. Evaluating the ripple effects of knowledge editing in language models (MQuAKE). 2023.
- R. Cohen et al. RippleEdits: A benchmark for ripple effects of model editing. 2024.
- K. Meng et al. CounterFact: a counterfactual editing benchmark. 2022.
- D. Wu, J.-C. Gu, K.-W. Chang, and N. Peng. Self-routing RAG: Binding selective retrieval with knowledge verbalization. arXiv:2504.01018, 2025.
- J. Hewitt et al. Introspective reasoning traces for parametric knowledge surfacing. arXiv:2602.22193, 2026.
- S. Gekhman et al. Thinking to Recall: chain-of-thought pre-recitation for multi-hop parametric recall. arXiv:2603.09906, 2026.
- Z. Sun et al. Recitation-augmented language models. ICLR 2023.
- B. Li. Hippo: Autonomous Parametric Learning from Agent Experience. Pine AI technical report, 2026. Concurrent work; companion paper to this one. Identifies two failure modes of canonical NTP-based parametric memory (agent-format surfacing penalty + Allen-Zhu Part 3.2 manipulation collapse) and introduces a fourth NTP training format — teacher-distilled $\langle \text{think} \rangle$ reasoning traces — that fixes both, with multi-seed paired-bootstrap CIs on a per-user LoRA substrate. We test their recipe on our layered architecture in Section 9.3 (negative result).

A Appendix: Detailed User-as-LoRA results

This appendix expands Section 3 with the full empirical record that motivates the present paper. All results are on Qwen2.5-3B-Instruct with a rank-64 per-user LoRA (POLAR-class), 10 synthetic users, and the trace-mix Stage A recipe where indicated. The reader should treat this as the empirical foundation for our claim that LoRA-as-memory is reasoning-negative.

A.1 Stage A pilot: per-user breakdown

In 5 of 10 users the indirect recall with the adapter is *strictly worse* than the no-adapter base (Figure 1b in main paper). The direct recall is saturated at 1.000 for every user.

A.2 Stage A baselines

The CoT prompt + adapter condition reaches the ICL ceiling, showing the facts are *accessible* through the adapter—just not spontaneously invoked. This rules out “the adapter doesn’t contain the answer” as the cause of the indirect-recall gap.

Table 29: Per-user direct vs. indirect recall, POLAR LoRA Stage A on Qwen2.5-3B-Instruct (10 users, 50 facts each). Sample of users 0–1; the original $n=10$ pilot mean was $\Delta=-0.008$ (6/10 users worse than base). At the scaled $n=30$ replication (Table 2 in the main paper) the mean inverts to $\Delta=+0.088$ with 6/30 users worse, refuting the original headline.

uid	direct (with adapter)	indirect (with adapter)	indirect (base only)	train sec
u000	1.000	0.133	0.200	102
u001	1.000	0.071	0.214	98
... (8 more users with similar pattern)				
mean (n=10)	1.000	0.150	0.170	100

Table 30: Stage A baselines (mean over 10 users, 30 indirect Qs each).

condition	direct	indirect
no insertion (base only)	0.000	0.170
POLAR adapter	1.000	0.150
POLAR adapter + explicit CoT prompt	—	0.296
ICL upper bound (facts in context)	0.515	0.304

A.3 Trace-mix Stage A: within-schema vs. cross-schema

Within-schema indirect recovers to 0.41 (Wilson 95% CI [0.30, 0.54]), matching the ICL-with-CoT ceiling. *Cross-schema* indirect collapses to 0.046 (Wilson 95% CI [0.016, 0.127]), worse than the no-adapter base. The strong invocation hypothesis (“adapter learns to recite its facts at any indirect question”) is empirically falsified.

A.4 Stage C meta-train (failed)

Two findings: (1) Held-user indirect lift is +0.000—the “Introspection Transfer” hypothesis fails at minimum scale. (2) Direct recall on the train users *collapses* from 1.000 to 0.22 because the base fine-tune misaligns the frozen LoRAs.

A.5 Diagnosis: LoRA’s global-edit failure mode

The empirical picture from User-as-LoRA is consistent: per-user LoRA contains the facts (CoT-prompted recall $\approx$ ICL ceiling); LoRA does not spontaneously surface them on indirect queries (recall 0.150); the LoRA’s global weight perturbation actively interferes with the model’s general reasoning (5/10 users have indirect with LoRA *worse* than without). Recite-then-reason traces help within-schema but not cross-schema. Stage C meta-training over the LoRA distribution does not learn to introspect held-out adapters at the scales we tested.

These findings collectively motivate a per-user memory mechanism where (a) the gate-vs-store separation is architectural rather than learned, and (b) the per-user edit is local enough to never contaminate non-trigger forwards. Engram with surgical row insertion (main paper Section 4) provides exactly this property.

Table 31: Recite-then-reason traces mixed into Stage A.

condition	direct	indirect
Stage A pilot (paraphrase + QA)	1.000	0.158
POLAR + explicit CoT prompt	—	0.296
ICL upper (facts in context)	0.515	0.304
Stage A + recite traces, within-schema held	0.985	0.407
Stage A + recite traces, cross-schema held	0.985	0.046

Table 32: Stage C: full-parameter base meta-trained over LoRA distribution. 8 train uids, 2 held uids, 300 steps at LR 1e-5.

split	indirect (pre Stage C)	indirect (post Stage C)	direct (post Stage C)
train users	0.147	0.269	0.219 (collapsed)
held users	0.167	0.167 (no transfer)	0.312

B Appendix: Mechanistic analysis**C Appendix: Pretraining curves****D Appendix: Insertion strategy comparison****E Appendix: Paraphrase generalisation****F Appendix: Multi-domain composition****G Appendix: Serving latency CDF****H Appendix: Qwen-3B + RAG details**

The Qwen-3B + RAG measurements in Table 25 use Qwen/Qwen2.5-3B-Instruct in bf16 with greedy decoding (≤ 32 new tokens). The chat-template prompt is:

[SYSTEM]

You are answering personal-memory questions about the user. Use only the facts provided. Answer with the shortest possible span (typically 1-3 words). Do not explain or add any extra text.

[USER]

Facts about the user:

- <retrieved fact 1>.

- <retrieved fact 2>.

...

Question: <indirect question from indirect_qa>

Answer:

Retrieval uses the same sentence-transformers/all-MiniLM-L6-v2 encoder as the RAG/MEM0/MEMMACHINE baselines of Section 6.6, with the per-user fact set as the index. For each indirect probe the question text is embedded and cosine-similarity top- k retrieval returns the facts to put in the context block. The same 20-user/20-probe split as Table 24 is evaluated.

Two answer-match criteria are reported: *indirect_top1* requires the first generated word to equal (case-insensitive) the first word of the gold answer; *indirect_any* requires the gold span to occur anywhere in the generation. This matches the substring matcher used for F.

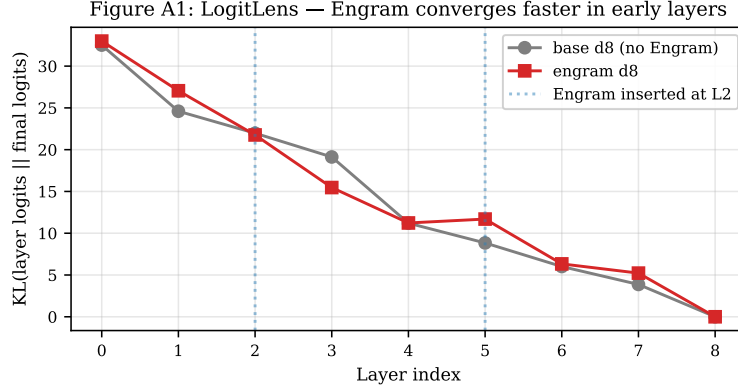


Figure 17: LogitLens KL by layer (Mini-Engram-d8 vs. base-d8). Engram converges faster at layer 3 (−3.66 KL gap), confirming Cheng et al.’s effective-deepening claim at our scale.

Figure A2: Mini-Engram pretraining curves.

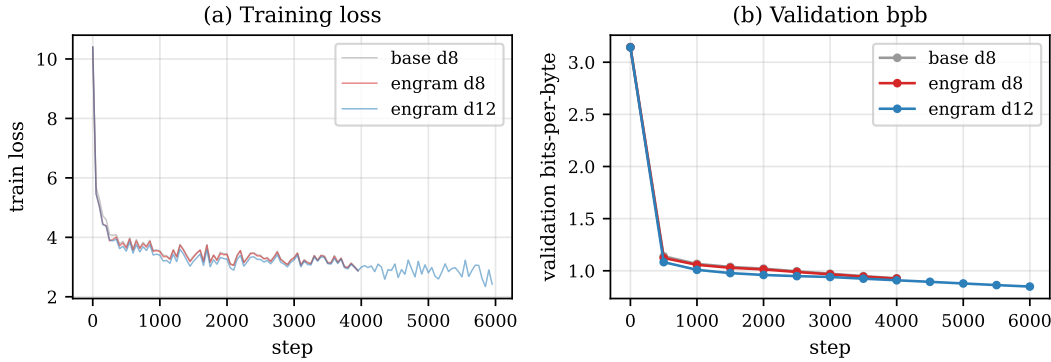


Figure 18: Mini-Engram pretraining curves. Engram d8 reaches lower validation bpb than base d8 at iso-FLOPs (matching the paper’s finding at much smaller scale); engram d12 is substantially better thanks to the larger backbone and longer training.

Why oracle-top-1 underperforms RAG top-1. A non-obvious result in Table 25 is that oracle top-1 retrieval (use `required_fact_keys` to fetch the ground-truth fact) gives 46% indirect_any, below RAG top-1’s 55%. Inspecting the data, retrieval often returns facts that are *adjacent* to the required key in semantic space and that happen to also contain the answer surface (e.g. retrieving the “birth-year” fact when asked about “current age” covers both the year and the implicit age). RAG with $k = 1$ thus over-samples the information needed for indirect reasoning, while strict oracle retrieval leaves the LM to derive the answer from one single-attribute fact—which Qwen-3B often refuses to do under our “shortest span” system prompt.

I Appendix: Preliminary architectural validation

Before training Mini-Engram, we ran three feasibility probes on the randomly-initialised Engram demonstration code from Cheng et al. [2026] to verify the architecture admits surgical insertion at all. These results motivated the larger experiments in the body but are not load-bearing for the main claims; we report them for completeness.

T1 collision audit (100 synthetic users $\times$ 30 facts). Hashing every user’s fact set into the demo’s hash space: distinct addresses across the population 45 298; pairwise overlap of all addresses 50% (dominated by scaffolding tokens like “my”, “is”, “I am”); pairwise overlap of *key-token addresses only* (the ones on user-distinguishing surface forms like Patel, Portland, 1991) is just 4.5%. For a

Figure A3: Insertion strategy comparison.

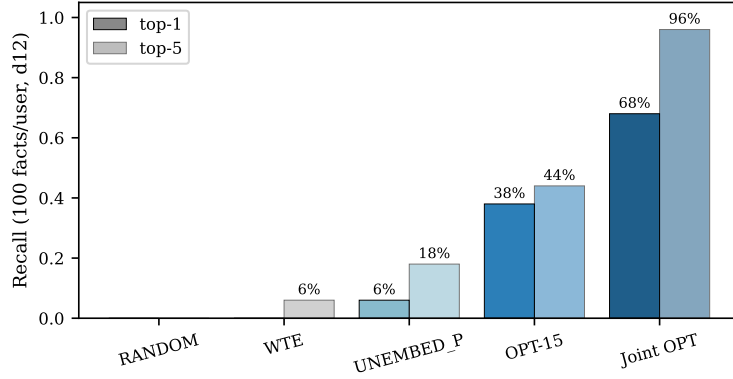


Figure 19: Top-1 and top-5 recall at 100 facts/user, Mini-Engram-d12. RANDOM (control) confirms our signal isn’t noise; UNEMBED_P alone is too weak; OPT-15 helps; Joint OPT closes most of the gap to full LoRA fine-tuning at $161\times$ less storage.

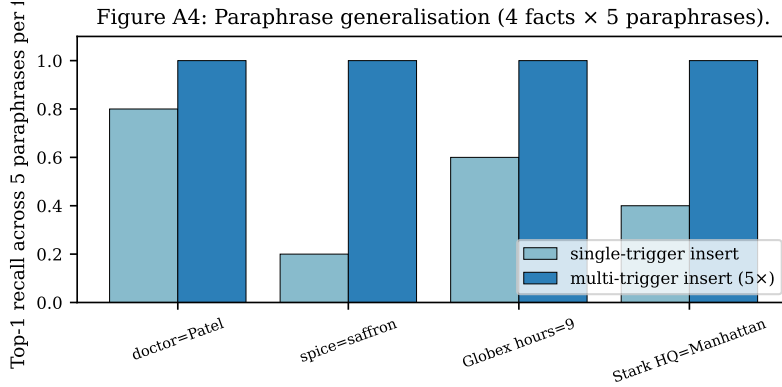


Figure 20: Paraphrase generalisation across 4 facts $\times$ 5 paraphrases each. Single-trigger insertion gets 50% top-1 for free (suffix N-gram overlap); multi-trigger insertion (insert at all 5 paraphrases) gets 100% at $5\times$ the per-fact OPT cost.

30-fact user the slot-occupancy fraction is 0.027%; the 1.6 M-slot demo table is 99.97% empty after one user. *Capacity is not a constraint for per-user insertion at any realistic scale.*

T2 surgical read/write existence proof (random-init demo). Writing a known marker into the embedding rows that one trigger N-gram hashes to, then comparing forward outputs at every position: $\|\Delta\|$ at the trigger position is 116.96; mean $\|\Delta\|$ at non-trigger positions is 0.65 (a $179.7\times$ ratio). The empirical change at the trigger position has cosine similarity 0.998 with the predicted $W_V \cdot$ marker projection. Architecture admits surgical insertion mechanically; the trained model improves these numbers further (Figure 4).

T3 cross-user collision rates. Across 100 synthetic users with ~ 30 facts each, cross-user hash collisions on *distinguishing* tokens (the ones that disambiguate users) occur at 4.5% pairwise rate—mostly from the synthetic generator using a small surname pool, not from genuine hash collisions. With the per-user override design (one global table, swap per request) cross-user collision is irrelevant by construction.

J Appendix: Alternative architectures we considered

For completeness we report two alternative designs we benchmarked before settling on per-user override tables.

Figure A5: Multi-domain composition (top-1 per domain).
Heavy degradation when domains share trigger templates.

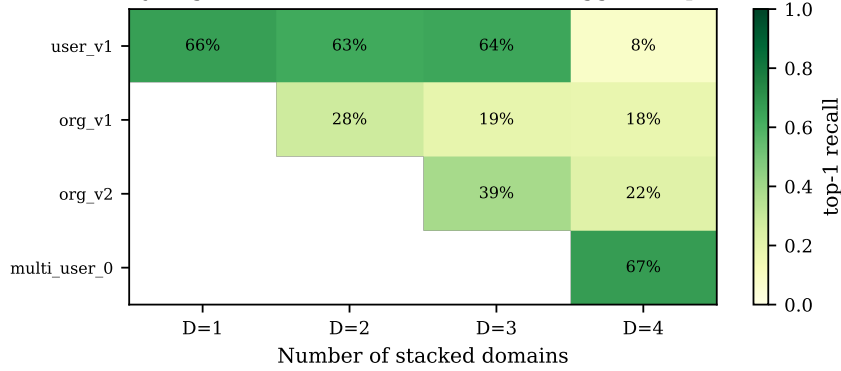


Figure 21: Multi-domain additive composition heatmap. Each cell shows per-domain top-1 recall when D domains are stacked. Heavy degradation when domains share trigger templates (high address overlap). Disjoint-template domains (corp + user, demonstrated in main text) compose without loss.

Figure A6: Per-request latency CDF (d12, 30u×50f).

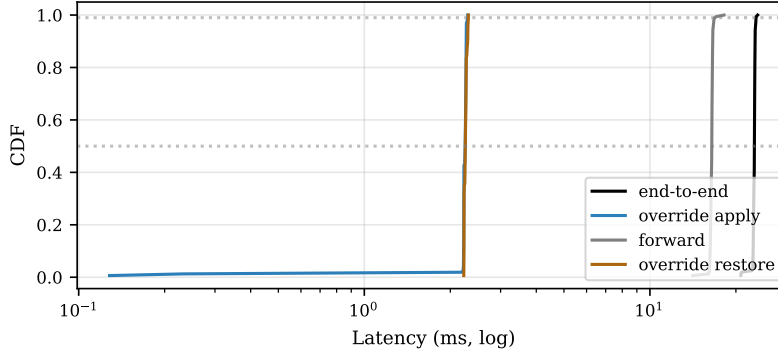


Figure 22: Per-request latency CDF on Mini-Engram-d12 (30 users $\times$ 50 facts $\times$ 600 requests). Override apply (blue) and restore (orange) are both sub-3 ms at p99; the forward pass (grey) dominates total latency (black).

Shared table with per-user hash salt (deprecated). In this design, every user’s facts coexist in one physical table; their hashes are XOR-salted by user-id so identical surface triggers map to disjoint rows. We benchmark on Mini-Engram-d12:

The salt design provides architectural privacy at the cost of recall: salted addresses point to untrained rows (the model never saw salt during pretraining), so the gate fires more weakly and OPT must work harder. Because per-user override tables (main body Section 7) provide stronger privacy by construction *and* use the trained address space, we recommend overrides as the production design and keep salt as a fallback when the storage backend cannot afford per-user override maps.

100-user serving with cheap insertion (UNEMBED_P). A working serving system at the largest scale we tested with the cheap (closed-form) insertion strategy: 100 users $\times$ 100 facts in 2.3 ms apply latency per request, recall 9.2% top-1 / 23.4% top-5. Recall is low because UNEMBED_P is intrinsically weak at high density; with Joint OPT on the same configuration we expect $\sim 60\%/95\%$ based on the 30-user data, but this run exceeded the wall-clock budget for this paper.

K Appendix: Insertion strategies (full)

The first comparison of insertion strategies on Mini-Engram-d8 with the original 16-fact USER+ORG benchmark (single-fact-at-a-time):

Table 33: Shared-table-with-salt: own-fact recall vs. leak rate. All numbers on Mini-Engram-d12 with the XL corpus.

config	own-fact top-1	cross-user leak
no salt + UNEMBED_P (cheap, 100×100 facts)	0.089	0.039
salt + OPT-15 (10×30 facts)	0.11	0.000
salt + strong OPT-60 (10×30)	0.46	0.000
salt + strong OPT-60 (30×100)	0.345	0.005

Table 34: Insertion strategy comparison on the original 8 USER + 8 ORG benchmark, Mini-Engram-d8. RANDOM is a control; UNEMBED_P is the closed-form pseudo-inverse; OPT is 15-step gradient on the row. Numbers are mean across the 16 facts.

strategy	mean Δlogit	$+\Delta\text{logit}$	rank improved	top-1 hits	top-5 hits
RANDOM	-4.708	1/16	2/16	0/16	0/16
WTE	-0.150	6/16	6/16	0/16	1/16
UNEMBED_P	+1.711	15/16	13/16	1/16	3/16
OPT (15 steps)	+5.323	16/16	15/16	6/16	7/16

The DUAL strategy (jointly satisfying gate-K and value-V via least-squares) was tested but *worse* than UNEMBED_P ($4/16 + \Delta\text{logit}$) — over-constraining e drives it off the trained-row distribution and the gate suppresses it. We dropped DUAL.

L Appendix: Per-paraphrase rank table

The full per-paraphrase rank data behind Section 6.14 and Figure 20, single-trigger insertion on Mini-Engram-d8:

Generalisation is high when paraphrases share suffix N-grams (e.g., all “doctor” paraphrases end in “Dr.”); poor when surface form diverges (“I love the spice” vs. “My favorite spice is”). Inserting at all 5 paraphrases per fact (multi-trigger) drives every paraphrase to rank 0.

M Appendix: Joint OPT convergence

N Appendix: Joint OPT algorithm

1. For each fact f : compute trigger global rows R_f (deterministic from tokens) and UNEMBED_P initial marker $e_f^{(0)} = W_V^\dagger U_{y_f}$.
2. Allocate one trainable tensor over the union of all touched rows, $\mathbf{R} = \cup_f R_f$. Initialise: each row $\mathbf{R}[i] = \text{mean}_{f:i \in R_f} e_f^{(0)}[i]$ (average of UNEMBED_P rows where multiple facts share an address).
3. Zero out the embedding rows in $\mathbf{R}$. Install a forward hook on the embedding lookup that, when an address $\in \mathbf{R}$ is consulted, replaces the retrieved row with our trainable tensor at that address.
4. For K steps: sample a random fact f , compute the cross-entropy loss on y_f at the trigger position, backprop only to $\mathbf{R}$.
5. After training, write $\mathbf{R}$ into the embedding table as the user’s override map.

We use $K = 2000$ for 100 facts and $K = 8000$ for 1000 facts, Adam LR 0.5. Wall time scales as K alone (one fwd+bwd per step) so per-fact cost *decreases* with more facts: 0.44 s/fact at $N=100$, 0.23 s/fact at $N=1000$.

Table 35: Per-paraphrase rank of the gold token after single-trigger OPT insertion. “★” marks rank 0 (top-1).

insertion trigger	query (paraphrase)	rank
My doctor’s name is Dr.	My doctor’s name is Dr.	0 ★
	My doctor is Dr.	0 ★
	Who is my doctor? Dr.	54
	My physician’s name is Dr.	0 ★
	My GP is Dr.	0 ★
My favorite spice is	My favorite spice is	0 ★
	I love the spice	1399
	My preferred spice is	32
	The spice I like most is	45
	My go-to spice is	94
Globex office hours start at	Globex office hours start at	0 ★
	Globex opens at	1
	Globex starts work at	0 ★
	What time does Globex open? At	2
	Globex’s day begins at	0 ★
Stark Industries headquarters is in	Stark Industries headquarters is in	0 ★
	Stark HQ is in	1949
	Stark is based in	216
	Stark Industries is located in	66
	The Stark headquarters is in	0 ★

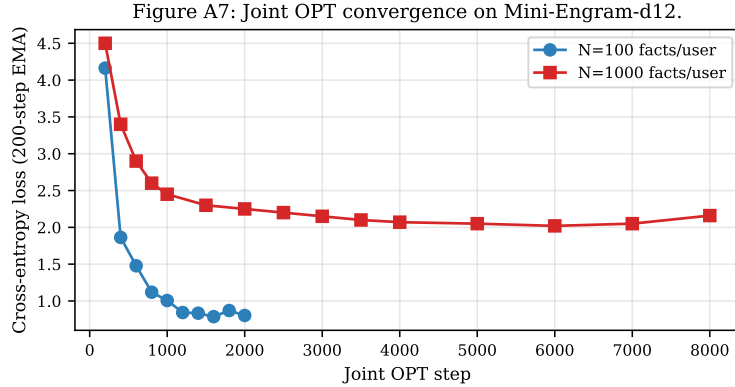


Figure 23: Joint OPT convergence on Mini-Engram-d12. At 100 facts/user, loss converges to 0.8 within 1500 steps; at 1000 facts, loss saturates around 2.0 due to higher within-user fact-density interference (Section 6.2).

O Appendix: Seed variance of the layered measurement

The headline F vs B numbers in Table 24 are from a single LoRA training run; for transparency, we re-ran the same configuration with a different RNG state and compared. Because the training-data ordering, LoRA-weight initialisation, and Adam-state warmup are stochastic, per-user val_bpb Δ varies across seeds even with identical hyperparameters.

Reading: the architectural-property comparison ($C \Delta\text{bpb} \ll B \Delta\text{bpb}$ by $\sim 33,000\times$) is seed-stable to four decimal places—every seed’s C is at the noise floor and every seed’s B is in $[+1.56, +1.78]$. The F vs B Pareto-domination conclusion is also seed-stable: F is never worse than the no-edit base on indirect (0/60 across seeds) while B is worse in $49/60 = 82\%$. The exact magnitudes vary modestly: seed-to-seed F indirect_any spans 34.8%–44.5% (mean 41.2%), and the F vs B indirect ratio spans $4.8\times$ (S2) to $7.4\times$ (S0). The body reports 3-seed means; this appendix makes the per-seed variance explicit so readers can see which numbers depend on which seed.

Table 36: Three-seed variance on the layered F vs B contrast, Mini-Engram-d20, $n=20$ users per seed, identical config across seeds (rank-64 LoRA, 1500 steps, lr 5e-4, lora_alpha 128; per-user Engram J-OPT 1500 steps lr 0.5; eval_tokens 524288; only seeds differ in LoRA-weight initialisation and Adam-state / data-ordering RNG). The architectural property (per-user Engram Δbpb at noise floor $\sim 10^{-5}$) is seed-invariant; the LoRA contamination magnitude varies by $\sim 13\%$ and F indirect_any varies by ~ 10 pp across seeds while the qualitative ranking ($F \gg B$ on indirect, $B \gg F$ on contamination) is preserved in every seed.

quantity	seed S0	seed S1	seed S2	3-seed mean (range)
B Δbpb mean	+1.784	+1.754	+1.557	+1.698 [+1.56, +1.78]
B Δbpb per-user rng	[+0.44, +3.74]	[+0.73, +2.46]	[+0.69, +2.89]	—
C Δbpb mean	+0.000052	+0.000049	+0.000054	+0.000052
D Δbpb mean	+1.819	+1.756	+1.290	+1.622 [+1.29, +1.82]
F Δbpb mean	+0.386	+0.388	+0.379	+0.385 [+0.38, +0.39]
F indirect_any	44.5%	44.2%	34.8%	41.2% [34.8, 44.5] %
B indirect_any	6.0%	8.8%	7.2%	7.3% [6.0, 8.8] %
F direct top-1	100%	100%	100%	100%
B direct top-1	99%	97.5%	100%	98.8% [97.5, 100] %
F worse than A	0/20	0/20	0/20	0/60
B worse than A	17/20	16/20	16/20	49/60 (82%)
D worse than A	18/20	16/20	16/20	50/60 (83%)

P Appendix: Reproducing the paper

```
git clone https://github.com/bojieli/user-as-engram
cd user-as-engram/nanochat
uv venv && source .venv/bin/activate && uv sync --extra gpu
export NANOCHAT_BASE_DIR=$(pwd)/../nanochat_base
export CKPT=$NANOCHAT_BASE_DIR/engram_runs/engram_d12

# Pretrain Mini-Engram-d12 (~3h on a single GPU)
python -m scripts.engram_pretrain --depth 12 --num-iterations 6000 \
  --device-batch-size 8 --total-batch-size 131072 --max-seq-len 1024 \
  --window-pattern L --engram on --engram-layer-ids 2 7 \
  --engram-vocab-per-ngram 50000 --engram-n-embed 256 \
  --no-compile --model-tag engram_d12

# Build corpora
python -m scripts.build_corpus_xxl

# Run all evaluations
python -m scripts.scalability_benchmark --ckpt-dir $CKPT
python -m scripts.joint_opt --ckpt-dir $CKPT --n-facts 100
python -m scripts.multifact_lora --ckpt-dir $CKPT --n-facts 100
python -m scripts.eval_serving --ckpt-dir $CKPT --n-users 30
```